Adaptive-adversary decremental approximate weighted matching in general graphs

Aditi Dudeja*

Department of Computer Science, University of Salzburg

arxiv:2312.08996v1; CC BY 4.0

Editorial note. Only thm:main counts. Complete author weighted/adaptive-adversary model, full epsilon dependence and all structural/dynamic arguments retained. Inputs/bbl mechanically expanded and abstract environment made explicit without changing its text; native originals preserved.

Editorial note. Optional footnote-placement decoration uses the standard article behavior because footmisc is unavailable; footnote text is retained.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

In this paper, we consider the problem of maintaining a $(1-\varepsilon)$-approximate maximum weight matching in a dynamic graph G, while the adversary makes changes to the edges of the graph. In the fully dynamic setting, where both edge insertions and deletions are allowed, Gupta and Peng [GP13] gave an algorithm for this problem with an update time of $\tilde{O}_\varepsilon(\sqrt{m})$.

We study a natural relaxation of this problem, namely the decremental model, where the adversary is only allowed to delete edges. For the cardinality version of this problem in general (possibly, non-bipartite) graphs, [ABD22] gave a decremental algorithm with update time $O_\varepsilon(\mathrm{poly}(\log n))$. However, beating $\tilde{O}_\varepsilon(\sqrt{m})$ update time remained an open problem for the weighted version in general graphs.

In this paper, we bridge the gap between unweighted and weighted general graphs for the decremental setting. We give a $O_\varepsilon(\mathrm{poly}(\log n))$ update time algorithm that maintains a $(1-\varepsilon)$ approximate maximum weight matching under adversarial deletions.[1] Like the decremental algorithm of [ABD22], our algorithm is randomized, but works against an adaptive adversary. It also matches the time bound for the cardinality version upto dependencies on ε and a $\log R$ factor, where R is the ratio between the maximum and minimum edge weight in G.

*aditi.dudeja@plus.ac.at.

[1] Independently and concurrently, Chen, Sidford, and Tu [CST23] obtained faster algorithms for decremental weighted matching in general graphs. Their work also improves on the state of the art for decremental unweighted matching in general graphs (that is, [ABD22]).

1 Introduction

In dynamic algorithms, the main goal is to maintain a key property of the graph while an adversary makes changes to the edges of the graph. An algorithm is called incremental if it only handles edge insertions, and decremental if it handles only edge deletions and fully dynamic if it handles both edge insertions and deletions.

The problem of maintaining a $(1-\varepsilon)$-approximation to the maximum matching in a dynamic graph is a well-studied one. In the fully dynamic setting, the best known update time is $O(m^{0.5-\Omega_\varepsilon(1)})$ (see [BKS23a]), and the conditional lower bounds proved in the [HKNS15, KPP16] suggest that this is a hard barrier to break through. Consequently, several relaxations of this problem have been studied. For example, one line of research has shown that we can get considerably faster update times if we settle for large approximation factors. Another research direction has been to consider more relaxed incremental or decremental models. In the incremental setting, there have been a series of upper and lower bound results (see [Dah16] and [Gup14]), culminating in the results of [GLS+19] (who give a $O(m \cdot (1/\varepsilon)^{O(1/\varepsilon)})$ total update time algorithm for $(1-\varepsilon)$-approximate matching in general graphs) and of [BK23] (who give a $O(m \cdot \text{poly}(1/\varepsilon))$ total update time algorithm for $(1-\varepsilon)$-approximate matching in bipartite graphs). In contrast, for decremental matching [BGS20, JJST22] gave an $O(m \cdot \text{poly}(\log n, 1/\varepsilon))$ total update time algorithm for $(1-\varepsilon)$-approximate matching in bipartite graphs, and [ABD22] extended this algorithm to the case of general graphs by giving an algorithm for $(1-\varepsilon)$-approximate decremental matching with total time $O_\varepsilon(m \cdot \text{poly}(\log n))$, thus completing the picture for approximate cardinality matching in the partially dynamic setting.

For the weighted case (in both fully and partially dynamic settings) the picture is less clear. [BDL21] showed that for the bipartite case, it is possible to reduce weighted matching to unweighted matching with an overhead of $(1-\varepsilon)$ in the approximation factor, and at a loss of a factor of $\log W$ and $(1/\varepsilon)^{O(1/\varepsilon)}$ in the update time. Recently, [BKS23b] showed that this exponential-in-$1/\varepsilon$ dependence was avoidable for bipartite graphs in the partially dynamic setting.

In contrast, for non-bipartite graphs, we don't have such a general reduction. Thus, while there has been significant recent progress on unweighted matching, the state-of-art for weighted matching lags far behind. Our main contribution in this paper, is to close this gap between weighted and unweighted matching for general graphs in the decremental dynamic setting:

Theorem 1. Let G be a weighted graph with weight function w on the edges of the graph and let $\varepsilon \in (0, 1/2)$. Then, there is a decremental algorithm with total update time $O_\varepsilon(m \cdot \text{poly}(\log n) \cdot \log R)$ (amortized $\tilde{O}_\varepsilon(1)$) that maintains an integral matching M with $w(M) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$, with high probability, where G refers to the current version of the graph. The algorithm is randomized but works against an adaptive adversary. The dependence on ε is $2^{(1/\varepsilon)^{O(1/\varepsilon)}}$.

In comparison, the decremental algorithm for unweighted $(1-\varepsilon)$ approximate matching, is also randomized but works against an adaptive adversary, and takes the same total time albeit without an additional loss of a $\log R$ factor and a dependence of $(1/\varepsilon)^{O(1/\varepsilon)}$ on ε.

1.1 Our Techniques

Several unweighted dynamic algorithms maintain a matching sparsifier, which is a sparse subgraph of G guaranteed to contain a large matching. As mentioned before, unlike in the bipartite case, there is no reduction from weighted to unweighted matchings in general graphs. So, the existing algorithms for weighted matchings in dynamic setting proceed by looking at weighted analogues of unweighted matching sparsifiers. For example, [BDL21] consider weighted versions of the EDCS and kernels which were first discovered in the context of cardinality matching problem (see [BS15, BS16, BHI15]). We use a similar approach as in [BDL21] and port the structural results of [ABD22] for unweighted graphs to the setting of arbitrary weighted graphs.

1.1.1 Structural Results for Decremental Matching

The congestion balancing framework of [BGS20] maintains a fractional matching that is robust to deletions. In order to ensure that the matching is robust and "spread out", they impose a capacity function κ on the edges of the graph. They repeatedly invoke a subroutine M-or-E*() that does one of the following (here $\mu(G)$ refers to the size of maximum cardinality matching):

(a) Either output a fractional matching $\vec{x}$, such that $\sum_{e \in E} x(e) \geqslant (1-\varepsilon) \cdot \mu(G)$ or,

(b) Output a set E^* of bottleneck edges along which we can increase capacities:

  (i) $\kappa(E^*)$ is $O(\mu \log n)$

  (ii) Every large matching in G has $\varepsilon \cdot \mu(G)$ edges in E^*.

The authors of [BGS20] use this subroutine to get an algorithm for decremental matching as follows: either M-or-E*() outputs a fractional matching $\vec{f}$ such that $\sum_{e \in E} \geqslant (1-\varepsilon) \cdot \mu(G)$, in which case we can use a black-box of [Waj20] to round this $\vec{f}$, or if there is no $\vec{f}$ satisfying this property, then it outputs a set of edges E^* along which we can increase capacity. Note that, (b)(i) ensures that the total capacity increase is small, and (b)(ii) ensures that we only increase capacity along edges necessary to form a large matching. Thus, they are able to show that the capacities remain small on average.

Predictably in general graphs, implementing such a subroutine is challenging because of the blossom constraints which impose the following roadblocks:

(a) An arbitrary fractional matching in a general graph can suffer from an integrality gap, which is not the case for bipartite graphs. Consequently, we need to be able to check if a given general graph has a large fractional matching obeying capacities and the blossom constraints.

(b) Once we know there is a large fractional matching obeying the above requirements, we need to efficiently find such a fractional matching. In bipartite graphs, this can be done use approximate max flows. However, for general graphs this correspondence between fractional matchings and flows is lost.

(c) Finally, in the case of bipartite graphs, if the fractional matching is small, the bottleneck edges E^* along which we have to increase capacity correspond to a min-cut in the graph, however for the case of general graphs, this characterization is not clear.

The main contribution of [ABD22] is to address these challenges through three structural lemmas, we state these informally, and then state our weighted versions of these lemmas.

Lemma 2. Let G be a graph and let $\mu(G)$ denotes the size of the optimal matching in G. Suppose κ is the capacity function on the edges of the graph, and let $\mu(G, \kappa)$ denote the optimal fractional matching of G, obeying capacity function κ, and all odd set constraints. Sample every edge e with $p(e) \propto \kappa(e)$ to create uncapacitated graph G_s. Then, $\mu(G_s) \geqslant \mu(G, \kappa) - \varepsilon \cdot n$.

The above lemma gives us a way to determine whether a given capacitated graph has a large fractional matching obeying blossom constraints by reducing this problem to the problem of finding an integral matching in a general graph. For the latter problem, there are efficient algorithms known. This takes care of Issue (a)

Lemma 3. Let G be a graph with capacity function κ on the edges of the graph. Let G_s be the graph obtained by sampling edges independently with probability $p(e) \propto \kappa(e)$. Consider a matching M of G_s such that $|M| \geqslant (1-\varepsilon) \cdot \mu(G_s)$. Let $M_H \subseteq M$ be the set of edges of high capacity. Let $V_H = V(M_H)$, Let $V_L = V \setminus V_H$. Compute a fractional matching $\vec{f}$ on the low capacity edges of $G[V_L]$, such that $\vec{f}$ obeys κ. Then, $|M_H| + \sum_{e \in E} f(e) \geqslant \mu(G_s) - \varepsilon \cdot n$.

Thus, by using integral matching algorithms, and approximate maximum flow, the above lemma helps us deal with Issue (b).

Lemma 4. The set of bottleneck edges of E^* can be identified by considering the edges of G that are not covered by the duals of the matching program of G_s.

The approximate matching algorithms given in the literature, also give us near feasible dual solution (see [DP09]), thus, the above lemma, in combination with these algorithms gives us a way to deal with Issue (c).

1.2 Weighted Analogues of Structural Results for Decremental Matching

Similar to the algorithm M-or-E*() in the unweighted decremental setting, we design an algorithm WeightedM-or-E*() that runs in time $O(m)$, and does one of the following two things.

(a) Either output a fractional matching $\vec{x}$, such that $\sum_{e\in E} w(e)\cdot x(e) \geqslant (1-\varepsilon)\cdot \mathsf{mwm}(G)$.

(b) Output a set E* of bottleneck edges along which we can increase capacities:

  (i) $\sum_{e\in E^*} \kappa(e)\cdot w(e) = O(\mathsf{mwm}(G)\cdot \log n)$

  (ii) Every large matching in G has $\varepsilon\cdot\mathsf{mwm}(G)$ weight in E^*.

The informal statements of the weighted versions of the unweighted lemmas are almost exactly the same:

Lemma 5. Let G be a graph and let $\mathsf{mwm}(G)$ denotes the weight of the maximum weight matching in G. Suppose κ is the capacity function on the edges of the graph, and let $\mathsf{mwm}(G,\kappa)$ denote the maximum fractional matching of G, obeying capacity function κ, and all odd set constraints. Sample every edge e with $p(e)\propto\kappa(e)$ to create uncapacitated graph G_s. Then, $\mathsf{mwm}(G_s)\geqslant \mathsf{mwm}(G,\kappa)-\varepsilon\cdot n$.

The proof of Lemma 2, relies on the Tutte-Berge Theorem ([Sch03]), and the main impediment to proving the above lemma is that there is no weighted analogue of this lemma. Thus, we have to rely on other structural probabilistic tools to come up with a proof.

Lemma 6. Consider a matching M of G_s such that $w(M)\geqslant(1-\varepsilon)\cdot\mathsf{mwm}(G_s)$. Let M_H be the set of edges of high capacity. Let $V_H = V(M_H)$, Let $V_L = V\setminus V_H$. Compute a weighted fractional $\vec{f}$ matching on the low capacity edges of $G[V_L]$, such that $\vec{f}$ obeys the capacity function κ, and the odd set constraints. Then, $\sum_{e\in E} f(e)\cdot w(e) + w(M_H)\geqslant \mathsf{mwm}(G_s)-\varepsilon\cdot n$.

The proof of the unweighted version of this theorem relies on the extended Hall's theorem, and similar to Tutte Berge theorem, there is no weighted version of this theorem. Thus, we have to come up with a generalization that relies on a primal-dual analysis.

Note that in the unweighted version, we can easily find such a fractional matching on low capacity edges, using approximate max-flow. However, weighted fractional matchings obeying capacities correspond to min-cost flow, and for this problem, we only have algorithms with $O(m^{1+o(1)})$ time (see [BGS20, vdBCK+23]) or ones where the runtime depends polynomially on the ratio between maximum and minimum capacities, which in our setting can be $O(n^2)$ (see [GHKT17]). Thus, we have to come up with a simple, efficient algorithm to do this task.

Lemma 7. There is an efficient algorithm to compute $(1-\varepsilon)$-approximate maximum weight fractional matching in a capacitated bipartite graph.

Finally, we show how to identify the set of bottleneck edges, E^*.

Lemma 8. The set of bottleneck edges of E* can be identified by looking at the dual of the matching program of G_s.

Notation Throughout the paper, we will use G to refer to the current version of the graph, and let V and E be the vertex and edge sets of G respectively. The graphs we deal with are weighted, and we use R to denote the ratio between the max-weight and min-weight edge. Additionally use $\mathsf{mwm}(G)$ to denote the weight of the maximum weight matching of G. During the course of the algorithm, we will maintain a fractional matching, which corresponds to a non-negative vector $\vec{x} \in [0,1]^m$ satisfying the following constraints : $\sum_{v \ni e} x(e) \leqslant 1$. For a set $S \subseteq E$, we let $x(S) = \sum_{e \in S} x(e)$. Given a capacity function κ on the edges of the graph, we say that $\vec{x}$ obeys κ if $x(e) \leqslant \kappa(e)$ for all $e \in E$. For a vector $\vec{x}$, we use $\mathrm{supp}(\vec{x})$ to denote the set of edges that are in the support of $\vec{x}$. For a fractional matching $\vec{x}$, we say that it satisfies odd set constraints if for all odd-sized sets $B \subseteq V$, we have, $\sum_{e \in G[B]} x(e) \leqslant \frac{|B|-1}{2}$. We use $\mathsf{mwm}(G, \kappa)$ to denote the size of the maximum weight fractional matching obeying the odd set constraints and the capacity function κ. Additionally, we will use $\gamma_\varepsilon = (1/\varepsilon)^{O(1/\varepsilon)}$.

Our main result is a decremental algorithm for maintaining $(1-\varepsilon)$-approximate maximum matching in weighted general graphs. In particular, we restate our main theorem.

Theorem 1. Let G be a weighted graph with weight function w on the edges of the graph and let $\varepsilon \in (0, 1/2)$. Then, there is a decremental algorithm with total update time $O_\varepsilon(m \cdot \mathrm{poly}(\log n) \cdot \log R)$ (amortized $\tilde{O}_\varepsilon(1)$) that maintains an integral matching M with $w(M) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$, with high probability, where G refers to the current version of the graph. The algorithm is randomized but works against an adaptive adversary. The dependence on ε is $2^{(1/\varepsilon)^{O(1/\varepsilon)}}$.

Some Tools from the Literature

In order to prove our theorem, we will need the following constant weight assumption due to [GP13]. This will enable us to only focus on solving the decremental matching problem on graphs with integral weights in $\{1, 2, \cdots, W\}$ as opposed to arbitrary weight graphs. We refer the reader to the Appendix C in [BDL21] for a proof of their reduction.

Lemma 9. Let G be a weighted graph with arbitrary weights with R being the ratio between maximum and minimum edge weights. Let $\varepsilon \in (0, 0.5)$. If there is an algorithm $\mathcal{A}$ that maintains an α-approximate maximum weight matching in a graph whose weights are $\{1, 2, 3, \cdots, W\}$ with update time $T(n, m, \alpha, W)$, then there is an algorithm $\mathcal{A}'$ that maintains a $(1-\varepsilon) \cdot \alpha$-approximate maximum weight matching in G in update time $O(T(n, m, \alpha, \gamma_\varepsilon) \cdot \log R)$.

Additionally, we need the following algorithm, which essentially states that in order to solve the problem on graphs with arbitrary weighted maximum matchings, it is sufficient to solve the problem on graphs which have a matching with a large weight. We state the following reduction which was used by [AKL19] in the stochastic setting for unweighted graphs. Here, we use it for the case of weighted graphs. We postpone the proof to the Section B.

Lemma 10. Let $\delta \in (0, 0.5)$ and G be a weighted graph with weights in $\{1, 2, \cdots, W\}$. Let $\mathsf{mwm}(G) \geqslant \mu$ and at most $(1+\delta) \cdot \mu$. There is an algorithm Vertex-Red(G, μ, δ) that in time $O(m \cdot W^3 \cdot \log n / \delta^4)$ returns $\lambda = \frac{100 \cdot \log n \cdot W^3}{\delta^4}$ multigraphs $H_1, \cdots, H_\lambda$ that have the following properties, where the second property holds with probability at least $1 - 1/n^2$.

(a) For all $i \in [\lambda]$, $|V(H_i)| = \frac{4 \cdot (1+\delta) \cdot W \cdot \mathsf{mwm}(G)}{\delta}$, $E(H_i) \subseteq E(G)$, and

(b) Suppose G is subject to deletions, which we simulate on each of the H_i's. More concretely, if an edge e is deleted from G, then its corresponding copy in H_i is also deleted. Suppose at the end of the deletion process, $\mathsf{mwm}(G) \geqslant \tilde{\mu}$ for some $\tilde{\mu} \geqslant (1-2\delta) \cdot \mu$, then $\mathsf{mwm}(H_i) \geqslant (1-\delta) \cdot \tilde{\mu}$ for some $i \in [\lambda]$.

We also need the following result about weighted matchings due to [DP14].

Lemma 11. There is an $O(m/\varepsilon \cdot \log 1/\varepsilon)$ time algorithm Static-Weighted-Match that takes as input a graph G, and returns a weighted integral matching such that $w(M) \geqslant \mathsf{mwm}(G) \cdot (1-\varepsilon)$.

Finally, since we are settling for an approximate matching, we can say the following:

Observation 12. We can assume that the ratio between the minimum and the maximum edge weight is at least ε^2/n^2. This is because throwing out all the edges below this threshold will only reduce the weight of the matching by $o(\varepsilon)$.

Assumption 13. In the rest of the paper we will assume that $\mathsf{mwm}(G) = \Omega(\log n)$. This is because the graph is undergoing deletions, and soon as $\mathsf{mwm}(G)$ drops below this value, we can switch to a different decremental algorithm which we describe in Section D.

Main Result

We first state the following lemma, and then assuming this show Theorem 1. Finally, we state the algorithm for that proves this lemma.

Lemma 14. Let G be a weighted graph with edge weights in $\{1, 2, \cdots, W\}$ and let $\varepsilon \in (0, 1)$. Then, there is a decremental algorithm with total update time $O(m \cdot 2^{W/\varepsilon^2} \cdot \text{poly}(\log n))$ that maintains an integral matching M with $w(M) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$, with high probability. The algorithm is randomized but works against an adaptive adversary.

Now, we use Lemma 14 to show Theorem 1.

Proof of Theorem 1. By the above lemma, we know that for integer-weighted graphs, with weights in $\{1, 2, \cdots, W\}$, there is a decremental $(1-\varepsilon)$-approximate maximum weight matching algorithm with a total update time of $O(m \cdot 2^{W/\varepsilon^2} \cdot \text{poly}(\log n))$. By applying Lemma 9, we know there is a decremental $(1-\varepsilon)$-approximate maximum weight matching algorithm for arbitrary weight graphs with maximum edge weight R, with an update time of $O(m \cdot 2^{(1/\varepsilon)^{O(1/\varepsilon)}} \cdot \text{poly}(\log n) \cdot \log R)$. □

Our next step is the following theorem, which we will use to prove Lemma 14.

Theorem 15. We are given a weighted multigraph $G = (V, E)$ with edge weights in $\{1, 2, \cdots, W\}$. Let $|V| = n$ and $|E| = m$. Suppose $\mathsf{mwm}(G) \geqslant \varepsilon \cdot n/100 \cdot W$ and $\mu \geqslant \mathsf{mwm}(G) \cdot (1-\varepsilon)$. Suppose G is subject to adversarial deletions. There is an algorithm Dec-Matching(G, μ, ε) that processes deletions in $O(m \cdot 2^{W/\varepsilon^2} \cdot \text{poly}(\log n))$ time, and has the following guarantees:

(a) When the algorithm terminates, $\mathsf{mwm}(G) < (1-2\varepsilon) \cdot \mu$. Upon termination, the algorithm outputs "no".

(b) Until the algorithm terminates, it maintains an integral matching M with $w(M) \geqslant (1-20\varepsilon) \cdot \mu$.

The decremental algorithm mentioned in Lemma 14 uses Dec-Matching() stated in Theorem 15 as a subroutine. We first show the proof of Lemma 14 assuming Theorem 15, and then we show the proof of Theorem 15.

Description of Algorithm for Lemma 14 The algorithm takes as input a graph G, and a parameter $\varepsilon > 0$. It first estimates the weight of the maximum matching of G by running Static-Weighted-Match(G, ε) given in Lemma 11. Let this estimate be μ. The algorithm then instantiates a procedure to create multiple parallel instances of the algorithm Dec-Matching(). As mentioned in Theorem 15, Dec-Matching() can maintain a $(1-\varepsilon)$-approximate maximum weight matching of a multigraph, provided the weight of the matching in the multigraph is $\Omega(n/W)$. So, we use the reduction mentioned in Lemma 10 to create λ multigraphs $H_1, \cdots, H_\lambda$ which have $O(W \cdot \mu/\varepsilon)$ vertices as opposed to n vertices, and we run parallel instances of Dec-Matching$(H_i, \mu \cdot (1-\varepsilon), \varepsilon)$ for all $i \in [\lambda]$. This procedure also instantiates a pointer called cur, which points to the least indexed H_i that contains a matching of weight at least $(1-\varepsilon) \cdot \mu$. The algorithm will output the matching indexed by cur (denoted M_{cur}). The algorithm then moves onto a procedure for deletion of edges. If an edge $e \in G$ is deleted, then it is deleted from each of the H_i's labelled active. If at this

Algorithm 2 Algorithm for Lemma 14

Input: Graph G and a parameter $\varepsilon > 0$

Output: A matching M with $w(M) \geqslant (1-8\varepsilon) \cdot \mathsf{mwm}(G)$

$M \leftarrow$ Static-Weighted-Match(G, ε).
$\mu \leftarrow w(M)$
$\lambda \leftarrow \frac{W^3 \cdot \log n}{\varepsilon^4 \cdot (1-\varepsilon)}$
procedure Instantiating Dec-Matching()
    $i \leftarrow 1$.
    cur$\leftarrow \lambda + 1$.
    Let $H_1, H_2, \cdots, H_\lambda$ be the multigraphs returned by Vertex-Red(G, μ, ε).
    Label $H_1, \cdots, H_\lambda$ as active.
    for $i \leqslant \lambda$ do
        if w(Static-Weighted-Match$(H_i, \varepsilon)) < (1-\varepsilon) \cdot \mu$ then
            Label H_i as inactive.
        end if
    end for
    for H_i labelled active do
        Initialize Dec-Matching$(H_i, \mu \cdot (1-\varepsilon), \varepsilon)$ (denoted $\mathcal{A}_i$).
        Let M_i be the matching maintained by $\mathcal{A}_i$.
    end for
    Let cur be the least index i such that H_i is active.
    if cur $> \lambda$ then
        terminate.
    end if
end procedure
procedure Handling deletion of edge e
    for H_i labelled active do
        Feed the deletion of e to $\mathcal{A}_i$.
        if $\mathcal{A}_i$ returns no then
            Label H_i as inactive.
        end if
    end for
    if H_{cur} labelled active then ▷ Label may be changed in Algorithm 1.
        Continue to output M_{cur}.
    else
        if cur $> \lambda$ then
            Return to Algorithm 1. ▷ Start over with new estimate for μ.
        else
            cur $\leftarrow$ cur $+ 1$
            Goto Algorithm 1.
        end if
    end if
end procedure

point, the weight of the maximum matching of H_{cur} drops by a significant amount, then we increment the value of cur. Finally, if all $\mathsf{mwm}(H_i)$ drops by a significant amount for all $i \in [\lambda]$, then we can conclude that $\mathsf{mwm}(G)$ has also dropped by a significant amount (this is due to contrapositive of Lemma 10(b)).

Proof of Lemma 14. We want to show how Algorithm 2 implies Lemma 14. We first start with the proof of correctness. We want to claim that Algorithm 2 indeed outputs a matching of size at least $(1-10\varepsilon)\cdot\mathsf{mwm}(G)$. To see this, observe that if we run Algorithm 2 with input G, ε then, we want to claim that it returns a matching M with $w(M) \geqslant (1-10\varepsilon)\cdot\mu$, where $\mu \geqslant (1-\varepsilon)\cdot\mathsf{mwm}(G)$. Consider the multigraphs $H_1, \cdots, H_\lambda$ output by Vertex-Red(G, ε). By Lemma 10, we know that these satisfy the requirements of Theorem 15. Thus, every run of Dec-Matching(H_i, μ, ε), until it returns no, it returns a matching of weight at least $(1-\varepsilon)\cdot\mu \geqslant (1-10\varepsilon)\cdot\mathsf{mwm}(G)$. This takes care of the correctness, and now we want to upper bound the runtime of the algorithm.

Claim 16. Each time Algorithm 2 executes Line 33, $\mathsf{mwm}(G)$ has dropped by a factor of $(1-2\varepsilon)$.

Proof. To see this, first observe that each time the algorithm returns to Algorithm 1, it must be the case that Dec-Matching$(H_i, \mu\cdot(1-\varepsilon), \varepsilon)$ has returned no for all $i \in [\lambda]$. Thus, by Theorem 15, we can conclude that $\mathsf{mwm}(H_i) < (1-2\varepsilon)\cdot(1-\varepsilon)\mu$ for all $i \in [\lambda]$. This in turn implies by contrapositive of Lemma 10(b) that $\mathsf{mwm}(G) < (1-2\varepsilon)\cdot\mu$ as well. Thus, $\mathsf{mwm}(G)$ has dropped from at least μ to at most $(1-2\varepsilon)\cdot\mu$. This concludes the proof of this claim. □

We upper bound the runtime of the algorithm as follows: We instantiate $\lambda = O\left(\frac{W^3\cdot\log n}{\varepsilon^4\cdot(1-\varepsilon)}\right)$ instances of Dec-Matching(). We start by bounding the runtime of the procedure that instantiates these λ instances. This has its runtime dominated by Vertex-Red(G, μ, ε), which takes $O(m)$ time. The second source of runtime is running the algorithm Static-Weighted-Match(H_i, ε), for $i \in [\lambda]$.

Next, we upper bound the runtime of the procedure that handles deletions. Note that if an edge e is deleted then, it is deleted from each of the λ copies. Thus, the total time to process deletions over m deletions is at most $O(m\cdot\lambda)$. Additionally, for maintaining each of the $\mathcal{A}_i$ for $i \in [\lambda]$, we take $O(m\cdot 2^{W/\varepsilon^2}\cdot\text{poly}(\log n)\cdot\lambda)$. Finally, by the above claim, we can conclude that Algorithm 1 is executed $O(\log n)$ times, so the total runtime of Algorithm 2 is $O(m\cdot 2^{W/\varepsilon^2}\cdot\text{poly}(\log n))$. □

2 Algorithm for Theorem 15

Before we give the algorithm for Theorem 15, we give some definitions. Recall, that is sufficient to design Dec-Matching() for the case of graph that have integer weights in $\{1, 2, \cdots, W\}$ by Lemma 9.

Definition 17. For a pair of vertices u, v, we define $D_i(u, v)$ to be the edges e between u and v that have weight i. Additionally, we also have $D(u, v) = \cup_{i=1}^{W} D_i(u, v)$. Since we assume integral weights, these sets are well-defined. If e is an edge between u, v, then $D_i(e) \coloneqq D_i(u, v)$ and $D(e) \coloneqq D(u, v)$.

Definition 18. Let G be a weighted multigraph, with n vertices and m edges. Let κ be the capacity function on the edges of the graph, and let $\vec{x}$ be a fractional matching. We define $\vec{x}^C$ to be a vector, with support size $\min\left\{\binom{n}{2}, m\right\}$, where for a pair of vertices u, v, $x_i^C(u, v) = \sum_{e\in D_i(u,v)} x(e)$. That is, $\vec{x}^C$ is obtained by "collapsing" all edges of the same weight between a pair of vertices together. Similarly, suppose $\vec{y}$ is a vector with $|\text{supp}(\vec{y})| \leqslant \binom{n}{2}\cdot W$, where, $y_i(u, v)$ is the entry corresponding to the edge of weight i between u and v. Then, we define $\vec{y}^D$ to be an m length vector such that for every $e \in E$ between u, v with $w(e) = i$, $y^D(e) \coloneqq \frac{y_i(u,v)\cdot\kappa(e)}{\kappa(D_i(u,v))}$. Intuitively, $\vec{y}^D$ is a multigraph obtained by distributing the $y_i(u, v)$ among all edges of the same weight.

Definition 19. Throughout the algorithm, we use $\alpha_\varepsilon \coloneqq 2^{W/\varepsilon^3}\cdot\log n$ and similarly, we have, $\rho_\varepsilon \coloneqq 2^{W/\varepsilon^2}\cdot\log n$.

We now state the main lemma, which will be used to show Theorem 15.

Lemma 20. Let G be a multigraph with edge weights in $\{1, 2, \cdots, W\}$. Then, there is an algorithm WeightedM-or-E*() that takes as input $G, \kappa, \varepsilon \in (0, 0.5)$, and a parameter $\mu \geqslant \mathsf{mwm}(G) \cdot (1 - \varepsilon)$, and in time $O(m \cdot W \cdot \log n / \varepsilon)$ returns one of the following.

(a) A fractional matching $\vec{x}$ such that $\sum_{e \in E} w(e) \cdot x(e) \geqslant (1 - 2\varepsilon) \cdot \mathsf{mwm}(G)$, with the following properties.

(i) Let $e \in D_i(u, v)$ such that $e \in \mathrm{supp}(\vec{x})$, with $\kappa(D_i(u, v)) \geqslant 1/\alpha_\varepsilon^2$, $x(D_i(u, v)) = 1$, and we have, $x(e) = \frac{\kappa(e)}{\kappa(D_i(u,v))}$. Moreover, since $\vec{x}$ is a fractional matching, we have $x(D_j(u, v)) = 0$ for all $j \neq i$.

(ii) Consider any $e \in D_i(u, v)$ such that $e \in \mathrm{supp}(\vec{x})$, with $\kappa(D_i(u, v)) \leqslant 1/\alpha_\varepsilon^2$, then $x(D_i(u, v)) \leqslant \kappa(D_i(u, v)) \cdot \alpha_\varepsilon$, and $x(e) \leqslant \kappa(e) \cdot \alpha_\varepsilon$. Moreover, for any $e \in D_j(u, v)$ with $\kappa(D_j(u, v)) > 1/\alpha_\varepsilon^2$, we have $x(e) = 0$.

(b) A set of edges E^* such that $\sum_{e \in E^*} w(e) \cdot \kappa(e) = O(\mathsf{mwm}(G) \cdot \log n)$, and for any integral matching M with $w(M) \geqslant (1 - 2\varepsilon) \cdot \mathsf{mwm}(G)$, we have $w(M \cap E^*) \geqslant \varepsilon \cdot \mathsf{mwm}(G)$. Additionally, for all $e \in E^*$, we have $\kappa(e) < 1$.

Consider the situation in which the algorithm WeightedM-or-E*() returns a fractional matching $\vec{x}$. For any pair of vertices u, v consider $\sum_{i=1}^{W} x_i^C(u, v)$, then this is either 1 or at most ε by Lemma 20(a). Thus, it satisfies odd set constraints for all odd sets of size at most $1/\varepsilon$, by a folklore lemma (Observation 71), we can then argue that $\vec{x}$ contains an integral matching of weight at least $(1 + \varepsilon)^{-1} \cdot \sum_{u \neq v} \sum_{i=1}^{W} x_i^C(u, v) \cdot i$ in its support.

Property 21. We will use WeightedM-or-E*() as a subroutine in Dec-Matching(), and we will get a matching $\vec{x}$ with the following properties. We mention these properties since they will help us visualize the fractional matching better.

(a) Each time WeightedM-or-E*() returns E^*, we will increase capacity along E^* by multiplying existing capacities by the same factor.

(b) Consider $u, v \in V$, and let e, e' be two edges between u and v such that $w(e) = w(e')$, then $\kappa(e) = \kappa(e')$ at all times during the run of the algorithm.

The next property follows directly as a consequence of Lemma 20(a).

Property 22. Let $\vec{x}$ be a matching output by Lemma 20, then $x(e) \leqslant \kappa(e) \cdot \alpha_\varepsilon^2$ for all $e \in E$. This is evident from Lemma 20(a).

Definition 23. Let G be a multigraph, and let $\vec{x}$ be a fractional matching of G. Then, we split $\mathrm{supp}(\vec{x})$ into two parts: $\vec{x}^i$ and $\vec{x}^f$, where, $\vec{x} = \vec{x}^i + \vec{x}^f$, and $\mathrm{supp}(\vec{x}^i) = \{e \mid x(D_j(e)) > 1/\alpha_\varepsilon^2 \text{ and } w(e) = j\}$ and $\mathrm{supp}(\vec{x}^f) = \{e \mid x(D_j(e)) \leqslant 1/\alpha_\varepsilon^2 \text{ and } w(e) = j\}$. When $\vec{x}$ is the output of WeightedM-or-E*(), then these correspond to the integral and fractional parts of $\vec{x}$.

Property 24. Let G be any multigraph, and let $\vec{x}$ be a fractional matching of G that is returned by WeightedM-or-E*(). Then, for any pair of vertices, u, v, we have the following properties, which are a consequence of Lemma 20(a)(i) and Lemma 20(a)(ii):

(a) Either $D(u, v) \cap \mathrm{supp}(\vec{x}^i) \neq \emptyset$ or $D(u, v) \cap \mathrm{supp}(\vec{x}^f) \neq \emptyset$, but not both.

(b) For any u, v, j such that $D_j(u, v) \cap \mathrm{supp}(\vec{x}) \neq \emptyset$, we have $D_j(u, v) \subseteq \mathrm{supp}(\vec{x})$.

Thus, the supports of $\vec{x}^i$ and $\vec{x}^f$ are vertex disjoint.

Property 25. Let $\vec{x}$ be a fractional matching returned by WeightedM-or-E*(), consider $\vec{z} = \vec{x}^i$. Then, $\vec{z}^C$ is an integral matching. This is implied by Lemma 20(a).

The next ingredient is a method to round weighted fractional matchings. This is implicit in [Waj20] (for unweighted, bipartite graphs), however in the Section E, we extend show this for the case of general, weighted graphs for the specific case of the fractional matching used by our algorithm.

Lemma 26. ([Waj20]) Suppose G is a integer-weighted multigraph, let $\varepsilon \in (0, 1/2)$, and let $\vec{x}$ be a fractional matching such that $x(D(e)) \leqslant \varepsilon^6$ for all $e \in E$. Then, there is a dynamic algorithm Sparsification$(\vec{x}, \varepsilon)$, that has the following properties.

(a) The algorithm maintains a subgraph $H \subseteq \text{supp}(\vec{x})$, such that $|E(H)| = O_\varepsilon(\mathsf{mwm}(\text{supp}(\vec{x})) \log n)$ and with high probability, $\mathsf{mwm}(H) \geqslant (1-\varepsilon) \cdot \sum_{e \in E} w(e) \cdot x(e)$.

(b) The algorithm handles the following updates to $\vec{x}$: the adversary can either remove an edge from $\text{supp}(\vec{x})$ or for any edge e, the adversary can reduce $x(e)$ to some new value $x(e) \geqslant 0$.

(c) The algorithm handles the above-mentioned updates in $\tilde{O}_\varepsilon(1)$ worst case time.

We now show how WeightedM-or-E*() and Sparsification() give us an algorithm for Theorem 15. This algorithm is called Dec-Matching().

Description of Dec-Matching() The algorithm takes as input a graph H, and instantiates its capacities. It then gets an estimate on $\mathsf{mwm}(H)$, denoted μ. The algorithm then runs the subroutine WeightedM-or-E*$(H, \mu, \varepsilon, \kappa)$, and this subroutine either outputs a fractional matching $\vec{x}$ with $\sum_{e \in E} w(e) \cdot x(e) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$, or if the matching is not heavy enough, a set of edges E^* (see Lemma 20). In the latter case, the algorithm proceeds by increasing the capacities along E^* (see Algorithm 3). On the other hand, if the fractional matching is large enough, then by Lemma 20(a), it can be broken up into two parts where $x^C((u,v)) = 1$ or $x^C((u,v)) \leqslant \varepsilon^6$ (see Algorithm 3). The algorithm Sparsification() extracts the fractional half of the matching (Algorithm 3). Note that this part satisfies the premise of Lemma 26, and therefore, the output is a sparse graph S, where $\mathsf{mwm}(S) \geqslant (1-\varepsilon) \sum_{e \in E} w(e) \cdot x(e)$. Thus, we then run Static-Weighted-Match(S, ε) and combine this with the integral part of $\vec{x}$ to output M (see Algorithm 3).

Subsequently, the algorithm then processes deletions. It maintains two counters, namely, CounterM and CounterX. As edges are deleted from the graph, at some point if CounterX increases to $\varepsilon \cdot \mu$, then we know that the adversary has reduced the weight of the fractional matching by a significant amount. At this point: either $\mathsf{mwm}(G)$ has dropped by a significant amount, or it hasn't and we must compute a fresh robust fractional matching, and therefore we begin a new phase. On the other hand, if CounterX has not increased, but CounterM has increased to $\varepsilon \cdot \mu$, then we know that $\vec{x}$ contains a high weight integral matching its support, and we recompute it.

Algorithm 3 Dec-Matching(H, μ, ε)

For all $e \in E(H)$, $\kappa(e) \leftarrow 1/\alpha_\varepsilon^{\lceil \log_{\alpha_\varepsilon} n \rceil}$
procedure Starting a new phase
    $\mu' \leftarrow$ Static-Weighted-Match(H, ε) ▷ $\mu' \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(H)$
    if $\mu' \leqslant (1-3\varepsilon) \cdot \mu$ then
        Return no, and terminate.
    else
        while WeightedM-or-E*$(H, \kappa, \varepsilon, \mu')$ returns E^* do
            For all $e \in E^*$, $\kappa(e) \leftarrow \kappa(e) \cdot \alpha_\varepsilon$
        end while
    end if
end procedure
Let $\vec{x}$ be the matching returned by WeightedM-or-E*(). ▷ This is a matching in a multigraph.
$\vec{y} \leftarrow \vec{x}^f$, $\vec{z} \leftarrow \vec{x}^i$ ▷ We will update $\vec{y}, \vec{z}$ as edges are deleted.
$S \leftarrow$ Sparsification$(\vec{y}^C, \varepsilon)$ ▷ Recall that Sparsification() is dynamic
$M \leftarrow$ Static-Weighted-Match$(S, \varepsilon) \cup \mathrm{supp}(\vec{z}^C)$. ▷ The matching M that is output.
CounterM $\leftarrow 0$ ▷ Counter for deletions to the integral matching.
CounterX $\leftarrow 0$ ▷ Counter for deletions to the fractional matching.
procedure For deletion of edge e from H
    if $e \in \mathrm{supp}(\vec{x})$ then
        Delete e from $\mathrm{supp}(\vec{x})$
        Update $\vec{z}^C$ accordingly. ▷ See Algorithm 3.
        Update $\vec{y}^C$ accordingly; Sparsification$(\vec{y}^C, \varepsilon)$ from Line 14 then updates S.
        CounterX $\leftarrow$ CounterX $+ w(e) \cdot x(e)$.
    end if
    if CounterX $> \varepsilon \cdot \mu$ then
        Goto Algorithm 3 ▷ End current phase and start new one.
    end if
    if $e \in M$ then
        Delete e from M
        CounterM $\leftarrow$ CounterM $+ w(e)$
    end if
    if CounterM $> \varepsilon \cdot \mu$ then ▷ The matching M is out of date
        CounterM $\leftarrow 0$
        $M \leftarrow$ Static-Weighted-Match$(S, \varepsilon) \cup \mathrm{supp}(\vec{z}^C)$ ▷ Recompute outputted matching M
    end if
end procedure

We now restate and prove Theorem 15.

Theorem 15. We are given a weighted multigraph $G = (V, E)$ with edge weights in $\{1, 2, \cdots, W\}$. Let $|V| = n$ and $|E| = m$. Suppose $\mathsf{mwm}(G) \geqslant \varepsilon \cdot n / 100 \cdot W$ and $\mu \geqslant \mathsf{mwm}(G) \cdot (1-\varepsilon)$. Suppose G is subject to adversarial deletions. There is an algorithm Dec-Matching(G, μ, ε) that processes deletions in $O(m \cdot 2^{W/\varepsilon^2} \cdot \mathrm{poly}(\log n))$ time, and has the following guarantees:

(a) When the algorithm terminates, $\mathsf{mwm}(G) < (1-2\varepsilon) \cdot \mu$. Upon termination, the algorithm outputs "no".

(b) Until the algorithm terminates, it maintains an integral matching M with $w(M) \geqslant (1-20\varepsilon) \cdot \mu$.

Proof of Theorem 15. We first argue that Algorithm 3 satisfies the properties of Theorem 15. First note that when the Algorithm 3 executes Algorithm 3, note that $\mu' \leqslant (1-3\varepsilon) \cdot \mu$. Since $\mu' \geqslant \mu(H)$, this implies

that $\mu(H) \leqslant (1-2\varepsilon)\cdot\mu$. This already shows (a). We now want to show (b). In order to show this, first note Lemma 20(a) states that $\sum_{e\in E} w(e)\cdot x(e) \geqslant (1-\varepsilon)\cdot \mathsf{mwm}(H)$. Note that the output matching M is a combination of $\mathsf{supp}(\vec{z}^C)$ and the integral matching computed on the output of Sparsification$(\vec{y}^C, \varepsilon)$. Recall that $\mathsf{supp}(\vec{z}^C)$ is a matching by Property 25. Also, by Property 24, these matchings are on disjoint set of vertices. Now we want to show that $w(M) \geqslant (1-\varepsilon)\cdot\mathsf{mwm}(G)$.

In order to see this, first note that $\vec{y}^C$ satisfies the premise of Lemma 26. Consequently, $\mathsf{mwm}(S) \geqslant (1-\varepsilon)\cdot\sum_{e\in E} w(e)\cdot y^C(e)$. Thus, we have the following line of reasoning:

$$\begin{aligned}
w(M) &\geqslant \sum_{e\in\mathsf{supp}(\vec{z}^C)} w(e) + w(\text{Static-Weighted-Match}(S,\varepsilon)) \\
&\geqslant \sum_{e\in\mathsf{supp}(\vec{z}^C)} w(e) + (1-\varepsilon)\cdot\mathsf{mwm}(S) \\
&\geqslant \sum_{e\in\mathsf{supp}(\vec{z}^C)} w(e) + (1-\varepsilon)^2\cdot\sum_{e\in E} w(e)\cdot y^C(e) \\
&\geqslant (1-2\varepsilon)\sum_{e\in E} w(e)\cdot x(e) \\
&\geqslant (1-3\varepsilon)\cdot\mathsf{mwm}(H)
\end{aligned}$$

We use the following lemma to bound the runtime of Dec-Matching(). We will give a formal proof of this lemma in Section 3.

Lemma 27. The subroutine WeightedM-or-E*() and Line 3 are called at most $O(\alpha_\varepsilon^3\cdot\log^2 n)$ times in Algorithm 3 during the course of m deletions.

Runtime Finally, we want to analyze the runtime of this algorithm. First, from Lemma 27 we can conclude that Algorithm 3 and WeightedM-or-E*() is called at most $O(\alpha_\varepsilon^3\cdot\log^2 n)$ many times. Moreover, from Lemma 20, we can conclude the total runtime of these routines is $O(m\cdot W\cdot\log n)$, thus the total runtime over all edge deletions is at most $O(m\cdot W\cdot\log^3 n\cdot\alpha_\varepsilon^3)$.

Next, we upper bound the number of phases. From Lemma 27, we can conclude that the number of phases is upper bounded by $O(\alpha_\varepsilon^3\cdot\log n)$. In every phase, we initialize the procedure Sparsification(). This takes $O(m\cdot\mathrm{poly}(\log n,\varepsilon^{-1}))$ time to initialize. Thus, the total time over all phases, is $O(m\cdot\mathrm{poly}(\log n,\varepsilon^{-1})\cdot\alpha_\varepsilon^3)$. Additionally, Sparsification() is dynamic, and it maintains the sparsifier in $O(\mathrm{poly}(\log n, 1/\varepsilon))$ update time. Finally, we also note that Algorithm 3 runs a static weighted matching algorithm on S each time the total weight of M drops by a $(1-\varepsilon)$ factor. This means, the graph has been subjected to at least $\frac{\varepsilon\cdot\mathsf{mwm}(G)}{W}$ deletions. Next, observe by Lemma 26, $|E(S)| = O(\mathsf{mwm}(H))$. Thus, from Lemma 11, we can conclude that Algorithm 3 takes amortized $O(W)$ time over all updates. □

The subsequent section is dedicated to the proof of Lemma 27.

3 Proof of Lemma 27

In order to prove the unweighted version of Lemma 27, [ABD22] and [BGS20] use the framework of congestion balancing. Here, we show that this framework extends to the case of general weighted graphs as well.

Definition 28. Let G be a multigraph, and let κ be the capacity function on the edges of the graph (if the adversary deletes an edge e, then let $\kappa(e)$ denote its capacity right before it is deleted). Let μ be the input to Dec-Matching(), when it is run on G,ε. Let $\mathcal{M} = \{M \mid w(M) \geqslant (1-\varepsilon)\cdot\mathsf{mwm}(G)\}$. Define the cost of an edge e, $c(e) = \log(n\cdot\kappa(e))$. For any integral matching M, define $\sum_{e\in M} c(e)\cdot w(e) = \sum_{e\in M} w(e)\cdot\log(n\cdot\kappa(e))$. We also define $\Pi(G,\kappa) = \min_{M\in\mathcal{M}}\sum_{e\in M} c(e)\cdot w(e)$. This corresponds to the min-cost matching in $\mathcal{M}$. If $\mathcal{M} = \emptyset$, then $\Pi(G,\kappa) = \infty$.

Observation 29. Initially, $\kappa(e) = 1/\alpha_\varepsilon^{\lceil 2\cdot\log_{\alpha_\varepsilon} n\rceil}$, so $\Pi(G,\kappa) = 0$. Thereafter, since capacities can only be increased, and edges can only be deleted, $\Pi(G,\kappa)$ can only increase.

Observation 30. When $\Pi(G,\kappa) = \infty$, then Line 5 of Algorithm 3 causes the algorithm to terminate.

Lemma 31. In Dec-Matching, we say that we have started a new phase, when $\varepsilon\cdot\mu$ weight of the fractional matching has been deleted, that is, CounterX has increased to $\varepsilon\cdot\mu$ (see Algorithm 3). Suppose we are in a phase where the algorithm does not terminate and let κ be the capacity function right before we process deletions, then, $\Pi(G,\kappa) = O(\mu\cdot\log n)$.

Proof. First observe that for any edge $e\in E$, $\kappa(e)\leqslant 1$, so $c(e)\leqslant\log n$. This is implied by the fact that E^* only increases capacity along edges with $\kappa(e)<1$ (see Lemma 20(b)), and due to the fact that initially, $\kappa(e) = 1/\alpha_\varepsilon^{\lceil 2\cdot\log_{\alpha_\varepsilon} n\rceil}$, and each time we increase $\kappa(e)$, we multiply it by a factor of α_ε. Moreover, we know that for any matching M, $w(M)\leqslant(1+\varepsilon)\cdot\mu$, since $\mu\geqslant(1-\varepsilon)\cdot\mathsf{mwm}(G)$. Thus, $\sum_{e\in M} w(e)\cdot c(e) = O(\mu\log n)$. This implies that $\Pi(G,\kappa) = O(\mu\log n)$. □

Definition 32. Let E_0 be the initial set of edges, then we let capacitated weight of E_0 to be defined as $w(\kappa(E_0)) = \sum_{e\in E_0} w(e)\cdot\kappa(e)$. If e is deleted by the adversary, then $\kappa(e)$ is the capacity of e right before deletion.

Lemma 33. Suppose a call to WeightM-or-E*$(G,\mu,\kappa,\varepsilon)$ returns the set E^* instead of a matching. Let κ' denote the new edge capacities after increasing capacities along E^*. Then,

(a) $w(\kappa'(E_0))\leqslant w(\kappa(E_0)) + \alpha_\varepsilon\cdot\mu\cdot\log n$, and

(b) $\Pi(G,\kappa')\geqslant\Pi(G,\kappa) + \mu/\varepsilon$.

Proof. We begin by recalling Lemma 20(b), which states that $w(\kappa(E^*)) = O(\mu\log n)$. Consequently, we have,

$$\begin{aligned} w(\kappa'(E_0)) &= w(\kappa(E_0\setminus E^*)) + w(\kappa'(E^*)) \\ &\leqslant w(\kappa(E_0)) + O(\mu\cdot\alpha_\varepsilon\cdot\log n) \end{aligned}$$

The last inequality follows from Lemma 20(b), and due to the fact that we multiply capacities along E^* by α_ε. To prove the second part of the claim, we have, again use the fact that every $M\in\mathcal{M}$ has the property that $w(M\cap E^*)\geqslant\varepsilon\cdot\mathsf{mwm}(G)$. Consequently, we have for every matching $M\in\mathcal{M}$, $c(w(M))$ increases by $\sum_{e\in M\cap E^*} w(e)\cdot\log\alpha_\varepsilon\geqslant\mathsf{mwm}(G)/\varepsilon$. Thus, the second part of the claim follows. □

Observation 34. The total number of calls to WeightedM-or-E*() (till Dec-Matching() terminates) that return E^* are upper bounded by $O(\varepsilon\cdot\log n)$. This is because each such call increases $\Pi(G,\kappa)$ by μ/ε, and by Lemma 31, we know that $\Pi(G,\kappa)$ is upper bounded by $\mu\log n$. Additionally, we can conclude that $w(\kappa(E_0))\leqslant\alpha_\varepsilon\cdot\mu\cdot\varepsilon\cdot\log^2 n$, and this is due to Lemma 33(a), and the fact that the number of calls to WeightedM-or-E*() that return E^* are at most $O(\varepsilon\cdot\log n)$ many.

Lemma 35. The total number of phases is at most $O(\alpha_\varepsilon^3\log^2 n)$. Therefore, the total number of times, a call to WeightedM-or-E*() returns a fractional matching is at most $O(\alpha_\varepsilon^3\log^2 n)$.

Proof. Note that we begin a new phase when CounterX grows from 0 to $\varepsilon\cdot\mu$. That is, the weight of the current fractional matching $\vec{x}$ has dropped by $\varepsilon\cdot\mu$. Let E_{del} be the set of edges that have been deleted from the multigraph. Let $\Phi_{\mathsf{del}} = \sum_{e\in E_{\mathsf{del}}}\kappa(e)\cdot w(e)$. This upper bounded by $w(\kappa(E_0))\leqslant\alpha_\varepsilon\cdot\mu\cdot\varepsilon\cdot\log^2 n$. Note that each time we begin a new phase, Φ_{del} increases by at least $\varepsilon\mu/\alpha_\varepsilon^2$ (one can conclude this since Property 22 holds). Thus, the total number of phases is at most $O(\alpha_\varepsilon^3\cdot\log^2 n)$. □

Proof of Lemma 27. There are two types of calls to the algorithm WeightedM-or-E*(): ones that return E^*, and the others which return a good fractional matching. From Observation 34 and Lemma 35, we can conclude that the total number of calls to WeightedM-or-E*() is at most $O(\alpha_\varepsilon^3\log^2 n)$. □

The rest of the paper is dedicated to giving an algorithm for WeightedM-or-E*(), and this culminates in a proof for Lemma 20, which is our main technical contribution.

4 Building Blocks for WeightedM-or-E*()

In this section, we prove the structural lemmas needed for each of the phases, and in order to do so, we need some probabilistic tools, for example Chernoff Bound:

Lemma 36 ([DP09]). Let $X_1, X_2, \cdots, X_k$ be k negatively correlated random variables, and let $X = \sum_{i=1}^{k} X_i$. Suppose $\mu = \mathbb{E}[X]$, and $\mu_{\min} \leqslant \mu \leqslant \mu_{\max}$. Then, we have,

$$\Pr\left(X \geqslant (1+\delta)\cdot\mu_{\max}\right) \leqslant \left(\frac{e^{\delta}}{(1+\delta)^{\delta}}\right)^{\mu_{\max}}.$$

Additionally, we will also need Bernstein's inequality.

Lemma 37 ([MMR94]). Let X be the sum of negatively associated random variables $X_1, \cdots, X_k$ with $X_i \in [0, M]$ for each $i \in [k]$. Then, for $\sigma^2 = \sum_{i=1}^{k} \mathrm{Var}[X_i]$ and all $a > 0$,

$$\Pr\left(X > \mathbb{E}[X] + a\right) \leqslant \exp\left(\frac{-a^2}{2\left(\sigma^2 + a\cdot M/3\right)}\right)$$

4.1 Phase 1 of WeightedM-or-E*()

Recall that we used $\mathsf{mwm}(G, \kappa)$ to denote the weight of the maximum weight fractional matching of G obeying κ as well as the odd set constraints. As in the congestion balancing step, we want to estimate $\mathsf{mwm}(G, \kappa)$. However, existing techniques only work for bipartite or general unweighted graphs. In this section, we extend this to the case of general weighted graphs. In particular, we extend the structural lemma to show that if in a weighted graph G, we sample every edge with probability $p(e) = \min\{1, \kappa(e)\cdot\rho_\varepsilon\}$ to create a graph G_s, then $\mathsf{mwm}(G_s) \geqslant \mathsf{mwm}(G, \kappa) - \varepsilon\cdot\mathsf{mwm}(G)$. Thus, now we can estimate $\mathsf{mwm}(G_s)$ by using existing results.

Lemma 38. Let G be an integer weighted multigraph with weights in $\{1, 2, \cdots, W\}$, and with $\mathsf{mwm}(G) \geqslant \max\{\varepsilon\cdot n/16\cdot W, \log n/\varepsilon^4\}$, where $\varepsilon \in (0, 1/2)$. Let κ be a capacity function on the edges of the graph, and let G_s be obtained by sampling every edge e independently with probability $p(e) = \min\{\kappa(e)\cdot\rho_\varepsilon, 1\}$. Let $\mathsf{mwm}(G, \kappa)$ be the weight of the maximum weight fractional matching of G that obeys the odd set constraints and the capacity constraints. Suppose $\mathsf{mwm}(G, \kappa) \geqslant (1-\varepsilon)\cdot\mathsf{mwm}(G)$, then, with high probability, $\mathsf{mwm}(G_s) \geqslant \mathsf{mwm}(G, \kappa) - \varepsilon\cdot\mathsf{mwm}(G)$.

Proof of Lemma 38. Let $\vec{x}$ denote the fractional matching that realizes $\mathsf{mwm}(G, \kappa)$. In order to prove the statement, we will construct a vector $\vec{z}$ in the support of G_s. This vector $\vec{z}$ will have the following properties: it will satisfy the fractional matching constraints and small blossom constraints with high probability. Additionally, it will also have the property that $\sum_{e\in E)} w(e)\cdot z(e) \geqslant (1-\varepsilon)\cdot\sum_{e\in E} w(e)\cdot x(e) \geqslant \mathsf{mwm}(G, \kappa) - \varepsilon\cdot\mathsf{mwm}(G)$ with high probability as well.

Since with high probability G_s, contains a fractional matching $\vec{z}$ satisfying the above properties, there is an integral matching M in support of G_s with $w(M) \geqslant (1-\varepsilon)\cdot\sum_{e\in E} w(e)\cdot z(e) \geqslant \mathsf{mwm}(G, \kappa) - \varepsilon\cdot\mathsf{mwm}(G)$ (see Observation 71).

Let X_e denote a random variable that takes value 1 if $e \in H_s$ and value 0 otherwise. We define $\vec{z}$ as follows:

$$z(e) = X_e \cdot \frac{x(e)}{\min\{1, \kappa(e)\cdot\rho_\varepsilon\}} \cdot (1-\varepsilon)$$

Note that $z(e)$ are independent random variables. The following is true.

$$\mathbb{E}\left[\sum_{e\in E} z(e)\cdot w(e)\right] = (1-\varepsilon)\cdot\sum_{e\in E} w(e)\cdot x(e)$$

Since this is a sum of independent random variables, using Chernoff bound (see Lemma 36), we have,

$$\Pr\left(\sum_{e\in E} z(e)\cdot w(e) \leqslant \sum_{e\in E} w(e)\cdot x(e) - 2\cdot\varepsilon\cdot \mathsf{mwm}(G)\right) = \exp\left(-\varepsilon^2 \mathsf{mwm}(G)/2\right)$$

By assumption, $\mathsf{mwm}(G) \geqslant 100\cdot\log n/\varepsilon^4$, we have that the above claim holds with probability at least $1 - O(1/n^{1/\varepsilon})$. We will now show that it obeys fractional matching constraints with high probability as well. Consider an edge e with $\kappa(e) > 1/\rho_\varepsilon$, we know that $e \in G_s$. Consequently, we have $\mathrm{Var}\left[z(e)\right] = 0$ for such an edge, since $z(e) = x(e)$ always. For any edge with $\kappa(e) < 1/\rho_e$, we have,

$$\mathrm{Var}\left[z(e)\right] \leqslant (1-\varepsilon)^2\cdot\left(\frac{x(e)}{\kappa(e)\cdot\rho_\varepsilon}\right)^2\cdot p(e) = (1-\varepsilon)^2\frac{x(e)^2}{\kappa(e)\cdot\rho_\varepsilon} \leqslant (1-2\varepsilon)\cdot\frac{x(e)}{\rho_\varepsilon}$$

Additionally, since $z(e)$ are independent random variables, we have $\mathrm{Var}\left[\sum_{v\ni e} z(e)\right] = \sum_{v\ni e}\mathrm{Var}\left[z(e)\right] \leqslant (1-2\varepsilon)\cdot\rho_\varepsilon^{-1}$. Additionally, we know that $\mathbb{E}\left[\sum_{e\ni v} z(e)\right] \leqslant (1-\varepsilon)$. Thus, we want to compute the probability of the event that $\sum_{e\ni v} z(e) \geqslant 1$. Note that, in order to do this, it is sufficient to consider the edges e for which $\kappa(e) < \rho_\varepsilon^{-1}$, since for edges other than this, $z(e) = x(e)$. Thus, we have, by Lemma 37, with $a = \varepsilon$, $M = \rho_\varepsilon^{-1}$,

$$\Pr\left(\sum_{e\ni v} z(e) > 1\right) \leqslant \exp\left(\frac{-\varepsilon^2}{\frac{1}{\rho_\varepsilon} + \frac{\varepsilon}{\rho_\varepsilon}}\right) = O\left(\frac{1}{n^{1/\varepsilon}}\right)$$

□

Taking a union bound over all vertices in the graph, we have our claim. Next, we want to compute the probability that $\vec{z}$ also satisfies small odd set constraints. To see this, consider any odd set B such that $|B| \leqslant 1/\varepsilon$. We know that by definition of $\vec{x}$, we have,

$$\sum_{e\in G[B]} x(e) \leqslant \frac{|B|-1}{2} \text{ and,}$$

$$\mathbb{E}\left[\sum_{e\in G[B]} z(e)\right] \leqslant (1-\varepsilon)\cdot\frac{|B|-1}{2}$$

Similarly, we can bound variance as well,

$$\mathrm{Var}\left[\sum_{e\in G[B]} z(e)\right] \leqslant (1-\varepsilon)\cdot\frac{|B|-1}{2}\cdot\frac{1}{\rho_\varepsilon}$$

Consider the following subclaim.

Observation 39. Let B be any odd set such that $3 \leqslant |B| \leqslant 1/\varepsilon$. Then,

$$\frac{|B|-1}{2} = (1-\varepsilon)\cdot\frac{|B|-1}{2} + \varepsilon\cdot\frac{|B|-1}{2} \geqslant (1-\varepsilon)\cdot\frac{|B|-1}{2} + \varepsilon$$

Let $\mathcal{E}_B$ be the event that $\sum_{e\in G[B]} z(e) \geqslant \frac{|B|-1}{2}$,

$$\begin{aligned}\Pr\left(\mathcal{E}_B\right) &\leqslant \Pr\left(\sum_{e\in G[B]} z(e) \geqslant (1-\varepsilon)\cdot\frac{|B|-1}{2} + \varepsilon\right)\\ &\leqslant \exp\left(-\frac{\varepsilon^2}{\frac{|B|-1}{2}\cdot\frac{1}{\rho_\varepsilon} + \frac{\varepsilon}{\rho_\varepsilon}}\right)\\ &= O\left(\frac{1}{n^{1/\varepsilon}}\right)\end{aligned}$$

The second equality follows from the fact that we are considering small blossoms, that is, $|B| \leqslant 1/\varepsilon$. Taking a union bound over all small blossoms, we have our claim.

4.2 Phase 2 of WeightedM-or-E*()

The algorithm WeightedM-or-E*() proceeds to Phase 2 only if in Phase 1, if $\mathsf{mwm}(G, \kappa) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$ (Lemma 38). In Phase 2, we now want to construct a good fractional matching.

Definition 40. Let G be a multigraph, we define $E_L = \{e \in E \mid e \in D_i(u,v), \kappa(D_i(u,v)) \leqslant 1/\alpha_\varepsilon^2\}$. Similarly, we define $\kappa^+(e) = \kappa(e) \cdot \alpha_\varepsilon$. Intuitively, these correspond to the low capacity edges.

Lemma 41. Let G be a weighted multigraph with maximum edge weight W. Suppose $\mathsf{mwm}(G) \geqslant \max\{\varepsilon \cdot n/16 \cdot W, \log n/\varepsilon^4\}$ Then, with high probability, for all $X \subseteq V$, we have

$$\mathsf{mwm}(G_s[X]) \leqslant \mathsf{mwm}(G[X] \cap E_L, \kappa^+) + \varepsilon \cdot \mathsf{mwm}(G)$$

Proof. Consider a fixed X, and from now on, we use $H \coloneqq G[X] \cap E_L$ and $H_s \coloneqq G_s[X] \cap E_L$. In order to prove this, we will make use of a primal-dual argument. Note that the problem of finding a $\mathsf{mwm}(H, \kappa^+)$ can be formulated as a linear program as follows.

$$\begin{aligned}
&\text{maximize} && \sum_{e \in E} w(e) \cdot x(e) && \\
&\text{subject to} && && \\
&\textstyle\sum_{e \ni v} x(e) && \leqslant 1 && \forall v \in X \\
&\textstyle\sum_{e \in G[B]} x(e) && \leqslant \tfrac{|B|-1}{2} && \forall B \in X_{\text{odd}} \\
&x(e) && \leqslant \kappa^+(e) && \forall e \in E
\end{aligned} \tag{1}$$

The corresponding dual program is as follows:

$$\begin{aligned}
&\text{minimize} && \sum_{u \in V} y_u + \sum_{B \subseteq X_{\text{odd}}} r(B) \cdot \left(\frac{|B|-1}{2}\right) + \sum_{e \in E(H)} z(e) \cdot \kappa^+(e) && \\
&\text{subject to} && && \\
& && y_u + y_v + z(e) + \sum_{B : (u,v) \in G[B]} r(B) \geqslant w(e) && \forall e \text{ between } u, v, \forall u, v \in X
\end{aligned} \tag{2}$$

Let $f(y, z, r)$ denote the optimal of the dual program, then by strong duality, we know that $\mathsf{mwm}(H, \kappa^+) = f(y, z, r)$. Similarly, for any uncapacitated graph we have the same primal and dual programs, except we don't have the third constraint in the primal program, and in the dual program we omit the z variables.

$$\begin{aligned}
&\text{maximize} && \sum_{e \in E} w(e) \cdot x'(e) && \\
&\text{subject to} && && \\
&\textstyle\sum_{e \ni v} x'(e) && \leqslant 1 && \forall v \in X \\
&\textstyle\sum_{e \in G[B]} x'(e) && \leqslant \tfrac{|B|-1}{2} && \forall B \in X_{\text{odd}}
\end{aligned} \tag{3}$$

The corresponding dual program is as follows:

$$\begin{aligned}
&\text{minimize} && \sum_{u \in V} y'_u + \sum_{B \subseteq X_{\text{odd}}} r'(B) \cdot \left(\frac{|B|-1}{2}\right) && \\
&\text{subject to} && && \\
&y'_u + y'_v + && \sum_{B : (u,v) \in G[B]} r'(B) \geqslant w(e) && \forall e \text{ between } u, v, \forall u, v \in X
\end{aligned}$$

Let $g(y', r')$ denote the optimal for dual program corresponding to H_s. Note that this is a random variable, since H_s is a random graph. We will show that with high probability,

$$g(y', r') \leqslant f(y, r, z) + \varepsilon \cdot \mathsf{mwm}(G)$$

By weak duality, we have, $\mathsf{mwm}(H_s) \leqslant g(y', r')$, and $f(y, r, z) = \mathsf{mwm}(H, \kappa^+)$. This will show our claim for a fixed H, and then we take a union bound over all H.

We will use $\left\{\{y_u\}_{u\in X}, \{r(B)\}_{B\in X_{\text{odd}}}\right\}$ to get a solution for the dual program for H_s. We will refer to this set of dual variables as an attempted cover. Let $E' = \{e \mid z(e) > 0\}$. Observe that the edges left uncovered by attempted cover of H_s are precisely a subset of E'. We now modify the cover as follows.

$$\begin{aligned}\Delta y_u &= \sum_{e\ni v, e\in H_s} z(e)\\ y'_u &= y_u + \Delta y_u\end{aligned}$$

Note that $\left\{\{y'_u\}_{u\in X}, \{r(B)\}_{B\in X_{\text{odd}}}\right\}$ is a valid cover of H_s. To see this, consider edge $e \in H_s$ and let u, v be the endpoints of H_s. Suppose $e \notin E'$, then,

$$\begin{aligned}w(e) &\leqslant y_u + y_v + z(e) + \sum_{e\in G[B]} r(B)\\ &\leqslant y_u + y_v + \sum_{e\in G[B]} r(B)\\ &\leqslant y'_u + y'_v + \sum_{e\in G[B]} r(B)\end{aligned}$$

Similarly, for an edge $e \in E'$, we have,

$$\begin{aligned}w(e) &\leqslant y_u + y_v + z(e) + \sum_{e\in G[B]} r(B)\\ &\leqslant y_u + y_v + \Delta y_u + \Delta y_v + \sum_{e\in G[B]} r(B)\\ &\leqslant y'_u + y'_v + \sum_{e\in G[B]} r(B)\end{aligned}$$

Moreover, $g(y', r') = f(y, r, z) + \sum_{u\in X} \Delta y_u$. Thus, it is sufficient to bound $\sum_{u\in X} \Delta y_u$.

$$\begin{aligned}\mathbb{E}\left[\sum_{u\in X} \Delta y_u\right] &= \sum_{e\in E} z(e) \cdot \Pr\left(e \in H_s\right)\\ &\leqslant \sum_{e\in E} z(e) \cdot \kappa(e) \cdot \rho_\varepsilon\\ &\leqslant \sum_{e\in E} \frac{z(e) \cdot \kappa^+(e)}{2^{W/\varepsilon^2}}\\ &\leqslant \mathsf{mwm}(H, \kappa^+) \cdot 2^{-W/\varepsilon^2}\end{aligned}$$

Using Chernoff and choosing $\delta = \frac{\varepsilon \cdot \mathsf{mwm}(G) \cdot 2^{W/\varepsilon^2}}{\mathsf{mwm}(H, \kappa^+)}$, we have,

$$\begin{aligned}\Pr\left(\sum_{u\in X} \Delta y_u \geqslant 2 \cdot \varepsilon \cdot \mathsf{mwm}(G)\right) &\leqslant \Pr\left(\sum_{u\in X} \Delta y_u \geqslant \mathsf{mwm}(H, \kappa^+) \cdot 2^{-W/\varepsilon^2} + \varepsilon \cdot \mathsf{mwm}(G)\right)\\ &\leqslant \exp\left(\varepsilon \cdot \mathsf{mwm}(G) - \varepsilon \cdot \mathsf{mwm}(G) \cdot \log\left(1 + \varepsilon \cdot 2^{W/\varepsilon^2}\right)\right)\\ &\leqslant \exp\left(-\varepsilon \cdot \mathsf{mwm}(G) \cdot \frac{W}{\varepsilon^2}\right)\end{aligned}$$

By assumption, $n \leqslant \frac{\mathsf{mwm}(G) \cdot W}{\varepsilon}$, thus taking a union bound over all subsets, we have our theorem. □

4.3 Phase 3: Finding Set E*

The algorithm WeightedM-or-E*() proceeds to Phase 3, if $\mathsf{mwm}(G, \kappa)$ is not large enough (see Lemma 38). At this point we have to increase capacity along some edges in the graph. Phase 3 finds such edges E^*, and makes sure it has that this set has the property that $\sum_{e \in E^*} w(e) \cdot \kappa(e) = O(\mathsf{mwm}(G) \log n)$, and moreover, every good matching in G has significant weight going through E^*.

We will first start with describing the dual program corresponding to the general matching LP:

$$\begin{aligned} \text{minimize} \quad & \sum_{u \in V} y_u + \sum_{B \subseteq V_{\text{odd}}} r(B) \cdot \left(\frac{|B| - 1}{2} \right) \\ \text{subject to} \quad & y_u + y_v + \sum_{B:(u,v) \in G[B]} r(B) \geqslant w(u,v) \quad \forall (u,v) \in E \end{aligned}$$

Additionally, we define $yr(e)$ to be the dual constraint corresponding to the edge e, and$f(y, r)$ to be the value of the objective function. We now describe some properties of Static-Weighted-Match(H, ε). We state these properties without proof for now and postpone the proof to the appendix.

Lemma 42. [DP14] There is an $O(m/\varepsilon \cdot \log 1/\varepsilon)$ time algorithm Static-Weighted-Match() that takes as input, a weighted graph G, and a parameter $\varepsilon > 0$, and outputs an integral matching M, and dual vectors $\vec{y}$ and $\vec{r}$ with the following properties.

(a) It returns an integral matching M such that $w(M) \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}(G)$

(b) A set Ω of laminar odd-sized sets such that $\{B \in V_{\text{odd}} \mid r(B) > 0\} \subseteq \Omega$.

(c) For all odd-sized sets B such that $|B| \geqslant 1/\varepsilon + 1$, $r(B) = 0$.

(d) For all $v \in V$, $y(v)$ is an integral multiple of ε, and for all B with $|B|$ odd, $r(B)$ is an integral multiple of ε.

(e) For each edge $e \in E$, we have $yr(e) \geqslant (1 - \varepsilon) \cdot w(e)$, that is e is approximately covered by $\vec{y}$ and $\vec{r}$.

(f) The value of the dual objective, $f(y, r)$ is at most $(1 + \varepsilon) \cdot \mathsf{mwm}(G)$.

We now state our first claim.

Claim 43. Suppose $H \subseteq G$, and $\vec{y}, \vec{r}$ are the dual vectors returned by Static-Weighted-Match(H, ε) and define $E_H = \{e \in E \mid yr(e) \geqslant (1 - \varepsilon) \cdot w(e)\}$. Let M be any matching of G, then $w(M \cap E \setminus E_H) \geqslant w(M) - (1 + \varepsilon)^2 \cdot \mathsf{mwm}(H)$.

Proof. Observe that if we scale up the dual variables $\vec{y}$ and $\vec{r}$ by $(1 + \varepsilon)$, then $\vec{y}$ and $\vec{r}$ is a feasible solution for the dual matching problem for the graph E_H. Thus, by weak duality and Lemma 42(f), we have $\mathsf{mwm}(E_H) \leqslant (1 + \varepsilon) \cdot f(y, r) \leqslant (1 + \varepsilon)^2 \cdot \mathsf{mwm}(H)$. This follows from Lemma 42(f). Thus, we have the following line of reasoning:

$$\begin{aligned} w(M) &= w(M \cap E_H) + w(M \cap E \setminus E_H) \\ &\leqslant \mathsf{mwm}(E_H) + w(M \cap E \setminus E_H) \\ &\leqslant (1 + \varepsilon)^2 \cdot \mathsf{mwm}(H) + w(M \cap E \setminus E_H). \end{aligned}$$

This proves the claim. □

We now describe the set E^*, and show that $w(\kappa(E^*)) = O(\mathsf{mwm}(G) \cdot \log n)$ with high probability.

Lemma 44. Let G be a multigraph such that $\mathsf{mwm}(G) \geqslant \varepsilon n / W$, and let κ be the capacity function on the edges of the graph. Suppose G_s is the graph obtained by sampling edge e with probability $p(e) = \min\{1, \rho_\varepsilon \cdot \kappa(e)\}$. Let $\vec{y}, \vec{r}$ be the duals returned by Static-Weighted-Match(G_s, ε). Let $E^* = \{e \mid yr(e) < (1 - \varepsilon) \cdot w(e)\}$, then with high probability, $w(\kappa(E^*)) = O(\mathsf{mwm}(G) \cdot \log n)$.

Proof. We consider the set $\mathcal{D}$ of duals $\vec{y}, \vec{r}$ that satisfy the following properties.

(a) For all $v \in V$, $y(v)$ is a multiple of ε, and for all $B \in V_{\text{odd}}$, $r(B)$ is a multiple of ε.

(b) Let $\Omega = \{B \mid r(B) > 0\}$, then Ω is laminar.

(c) If $r(B) > 0$ for some B, then $|B| \leqslant 1/\varepsilon$.

Observe that $\mathcal{D}$ contains all possible duals that could be returned by Static-Weighted-Match(). Now, we bound $|\mathcal{D}|$.

$$\begin{aligned}
|\mathcal{D}| &\leqslant \sum_{i=0}^{2n} \binom{n^{1/\varepsilon}}{i} \cdot \left(\frac{W}{\varepsilon}\right)^n \cdot \left(\frac{W}{\varepsilon}\right)^{2n} \\
&\leqslant 2n \cdot \binom{n^{1/\varepsilon}}{n} \cdot \left(\frac{W}{\varepsilon}\right)^n \cdot \left(\frac{W}{\varepsilon}\right)^{2n} \\
&\leqslant 2n \cdot \binom{n^{1/\varepsilon}}{n} \cdot \left(\frac{n^2}{\varepsilon^3}\right)^n \cdot \left(\frac{n^2}{\varepsilon^3}\right)^{2n} \\
&\leqslant 2^{10/\varepsilon^3 \cdot n \cdot \log n} \\
&\leqslant 2^{10/\varepsilon^3 \cdot \frac{\mathsf{mwm}(G)}{W} \cdot \log n}
\end{aligned}$$

This follows from the following argument. Since Ω is laminar (due to (b)), there are at most $2n$ sets contained in Ω, and from (c), we can conclude that these laminar sets are chosen from among $n^{1/\varepsilon}$ sets. Similarly, (a) suggests that each $y(v)$ and chosen $r(B)$ can be assigned W/ε values, thus for a given choice of Ω, there are at most $\left(\frac{W}{\varepsilon}\right)^n \cdot \left(\frac{W}{\varepsilon}\right)^{2n}$ choices for $\vec{y}$ and $\vec{r}$. We can derive the last set of equations by observing that $W \leqslant n^2/\varepsilon^2$ (see Observation 12)and by the premise of our lemma, we have that $\mathsf{mwm}(G) \geqslant \varepsilon n/W$. Additionally, observe that $\mathcal{D}$ includes the set of duals that can be returned by G, and therefore $|\mathcal{D}|$ is an upper bound on the set of possible duals returned by Static-Weighted-Match().

Now, consider any $\vec{y}, \vec{r} \in \mathcal{D}$, and let $E_{y,r} = \{e \mid yr(e) < (1-\varepsilon) \cdot w(e)\}$. Observe that for all $e \in E_{y,r}$, we have $\kappa(e) < 1/\rho_\varepsilon$, since otherwise $\kappa(e) = 1$, and e would be included in G_s with probability 1. Thus, for all $e \in E_{y,r}$, we have $\kappa(e) < 1/\rho_\varepsilon$. Now, $\vec{y}, \vec{r}$ are such that $w(\kappa(E_{y,r})) > \mathsf{mwm}(G) \cdot \log n$, then none of the edges in $E_{y,r}$ were sampled. Note that in this case, $\kappa(E_{y,r}) \geqslant \mathsf{mwm}(G) \cdot \log n \cdot W^{-1}$. Let $\mathcal{E}_1$ denote the event that $\vec{y}, \vec{r}$ are returned by Static-Weighted-Match(G_s, ε) and $\mathcal{E}_2$ be the event that $E_{y,r}$ is not sampled into G_s. Then, observe that in this case,

$$\begin{aligned}
\Pr(\mathcal{E}_1) &\leqslant \Pr(\mathcal{E}_2) \\
&\leqslant \prod_{e \in E_{y,r}} (1 - p(e)) \\
&\leqslant \exp\left(-\sum_{e \in E_{y,r}} \kappa(e) \cdot \rho_\varepsilon\right) \\
&\leqslant \exp\left(-\frac{\mathsf{mwm}(G)}{W} \cdot \rho_\varepsilon\right)
\end{aligned}$$

Now, in order to upper bound this, we take a union bound over all $|\mathcal{D}|$ many possible duals. From the previous discussion these we know that $|\mathcal{D}| \leqslant \exp\left(\frac{10}{\varepsilon^2} \cdot \frac{\mathsf{mwm}(G)}{W} \cdot \log n\right)$. This concludes the proof. □

4.4 Algorithm for Weighted-Frac-Match()

The final ingredient we need for WeightedM-or-E*() is the following lemma, which computes a fractional matching on low capacity edges.

Lemma 45. Consider a weighted bipartite multigraph G with edge capacity function κ, and edge weights in $\{1, 2, \cdots, W\}$. There is an algorithm that in $O(m \cdot W \cdot \log n \cdot 1/\varepsilon)$ time finds a fractional matching $\vec{x}$ obeying the capacity function κ such that $\sum_{e \in E} w(e) \cdot x(e) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G, \kappa)$.

In order to come up with this algorithm, we first recall the maximum weight capacitated fractional matching linear program for bipartite graphs, and its corresponding dual.

$$\begin{aligned} \text{maximize} \quad & \sum_{e \in E} w(e) \cdot x(e) \\ \text{subject to} \quad & x(e) \leqslant \kappa(e) \qquad \forall e \in E \\ & \sum_{e \ni v} x(e) \leqslant 1 \qquad \forall v \in V \end{aligned} \tag{4}$$

$$\begin{aligned} \text{minimize} \quad & \sum_{v \in L} y(v) + \sum_{u \in R} y(u) + \sum_{e \in E} z(e) \cdot \kappa(e) \\ \text{subject to} \quad & yz(e) \geqslant w(e) \qquad \forall e \in E \end{aligned} \tag{5}$$

Here, $yz(e) = y(u) + y(v) + z(e)$, where u, v are the two endpoints of e. Additionally, before stating our algorithm, we give some definitions.

Definition 46 (Residual Graph). Given a bipartite graph $G = (L, R, E)$, with a capacity function κ on the edges of the graph. Let $\vec{x}$ be a fractional matching obeying κ. We define G_x to be the residual graph with respect to $\vec{x}$. In particular, this is a directed graph, where corresponding to each undirected edge $e \in G$, there are two directed edges, e_f (forward edges, directed from L to R), and e_b (backward edge, directed from R to L). Moreover, e_f has a residual capacity of $\kappa(e) - x(e)$, and e_b has a capacity of $x(e)$. Thus, if an edge e is saturated, that is $x(e) = \kappa(e)$, then e_f is not in G_x. Similarly, if $x(e) = 0$, then e_b is not in G_x.

Definition 47 (Free Vertices). Given a graph G, and a fractional matching $\vec{x}$, we say that a vertex v is free with respect to $\vec{x}$ if $\sum_{e \ni v} x(e) < 1$.

Definition 48. An augmenting path in G_x, will refer to a path starting and ending in a free vertex, and all the intermediate vertices on this path will be saturated.

Before stating our algorithm, we will state the invariants that the algorithm will maintain. Subsequently, we will show that maintaining these invariants throughout implies that the algorithm will output a matching satisfying the approximation guarantees of Lemma 45 and in the desired runtime.

Remark 49. We round down ε so that $1/\varepsilon$ is an integer.

Property 50. Our algorithm will at all times maintain, a fractional matching $\vec{x}$, and the dual variables $y(u)$ for all $u \in V$, and $z(e)$, for all $e \in E$ with the following properties.

(a) Granularity: Throughout the algorithm, the duals $y(u)$ for all $u \in V$, and $z(e)$ for all $e \in E$, are multiples of ε.

(b) Domination: For all directed edges e_f and e_b in G_x, we have $yz(e_f) \geqslant w(e) - \varepsilon$, and $yz(e_b) \geqslant w(e) - \varepsilon$.

(c) Tightness: For all backward edges e_b, we additionally have the property that $yz(e_b) \leqslant w(e_b) + \varepsilon$.

(d) Free Duals: The duals for free vertices $u \in L$ are equal, and are at most the dual values for other vertices. The free vertex duals in R are always 0. The algorithm terminates when the free vertex duals in L are all 0.

(e) Complementary Slackness: If $z(e) > 0$ for some $e \in E$, then $x(e) = \kappa(e)$.

Definition 51 (Eligible Edges). Let $\vec{x}$ be the current fractional matching maintained by the graph, and let G_x denote the residual graph with respect to $\vec{x}$. A forward edge e_f is eligible if $yz(e_f) = w(e) - \varepsilon$, and backward edge e_b is said to be eligible if $yz(e_b) = w(e) + \varepsilon$. We use G_x^t to denote the eligible subgraph of G_x.

Lemma 52. The fractional matching $\vec{x}$ returned by the algorithm mentioned in Property 50 has $\sum_{e \in E} w(e) \cdot x(e) \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}(G)$.

Proof. In order to see this, consider the dual variables returned by the algorithm. Note that if we increase each of the duals by a factor of $(1+\varepsilon)$, then the dual solutions $\vec{y}, \vec{z}$ maintained by the algorithm are feasible dual solutions to the dual program (this is by Property 50(b)). Thus, by weak duality, we have,

$$\sum_{u \in V} y(u) + \sum_{e \in E} z(e) \cdot \kappa(e) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$$

Due to complementary slackness, (Property 50(e)), we have,

$$\sum_{u \in V} y(u) + \sum_{e \in E} x(e) \cdot z(e) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$$

Due to the fact that free vertex duals, at the end of the algorithm are zero (Property 50(d)), we have,

$$\sum_{u \in V} \sum_{e \ni u} x(e) \cdot y(u) + \sum_{e \in E} z(e) \cdot x(e) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$$

Simplifying the above expression, we have,

$$\sum_{e \in E} yz(e) \cdot x(e) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$$

Using Property 50(c), that is, tightness, we have,

$$\sum_{e \in E} (1+4\varepsilon) \cdot w(e) \cdot x(e) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$$

This shows the claim. □

We now want to state the precise algorithm.

Algorithm 4 Weighted-Frac-Matching(G, κ, ε)

Input: A bipartite multigraph G with capacity and weight edge functions κ and w.
Output: A fractional matching $\vec{x}$ with $\sum_{e \in E} w(e) \cdot x(e) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G, \kappa)$

1: procedure Initialization:
2: $x(e) \leftarrow 0$ for all $e \in E$ ▷ Initializing $\vec{x}$
3: $y(u) \leftarrow W - \varepsilon$ for all $u \in L$ ▷ Initializing left vertex duals.
4: $y(u) \leftarrow 0$ for all $u \in R$ ▷ Initializing right vertex duals.
5: $z(e) \leftarrow 0$ for all $e \in E$ ▷ Initializing edge duals.
6: end procedure
7: while y-values of free left vertices are greater than 0 do
8: for $e_b \in G_x^t$ with $z(e) > 0$ do
9: $z(e) \leftarrow z(e) - \min\{z(e), yz(e) - w(e) + \varepsilon\}$. ▷ Either $z(e) = 0$, or $e \notin G_x^t$ as $yz(e) = w(e) - \varepsilon$
10: end for
11: Update G_x^t
12: procedure Augmentation:
13: Find the maximal set $\mathcal{P}$ of augmenting paths in G_x^t.
14: Augment along $\mathcal{P}$. Update $\vec{x}$ and G_x^t.
15: end procedure
16: procedure Dual Adjustment:
17: Let Z be the set of vertices reachable from free left vertices in G_x^t.
18: For all ineligible edges e_b directed from $R \setminus Z$ to $L \cap Z$, with $yz(e) = w(e) - \varepsilon$, $z(e) \leftarrow z(e) + \varepsilon$.
19: For all $u \in L \cap Z$, $y(u) \leftarrow y(u) - \varepsilon$.
20: For all $u \in R \cap Z$, $y(u) \leftarrow y(u) + \varepsilon$.
21: end procedure
22: end while

We now show that Algorithm 4 proves Property 50. Later, we will show runtime guarantees of Algorithm 4. Towards this, we start with the following observation.

Observation 53. Let $e \in G$ be an edge, then e_f and e_b cannot simultaneously appear in G_x^t

Definition 54. Let H and G be directed, capacitated graphs with capacity functions c_h and c_g respectively. Then, we say $E(H) \subseteq E(G)$ if $e \in H$ implies $e \in G$, and $c_h(e) \leqslant c_g(e)$.

Claim 55. After Algorithm 4, there are no augmenting paths in the graph G_x^t.

Proof. Let $\vec{y}$ denote the fractional matching after the augmentation step. Then, in order to show the claim, it is sufficient to show $E(G_y^t) \subseteq E(G_x^t)$. Consider any forward edge $e_f \in E(G_x^t)$, observe that $e_b \notin E(G_x^t)$ due to Observation 53. Since we don't change the duals, $e_b \notin E(G_y^t)$ as well. Finally, e_f can either be augmented along or not, in either case, we have, $\kappa(e) - x(e) \geqslant \kappa(e) - y(e)$. Such an argument applies for any edge e_b as well.

From this, we can conclude that any augmenting path that is present in G_y^t is in G_x^t as well, and this contradicts the fact that we found a maximal set of augmenting paths $\mathcal{P}$. □

We now show that granularity property holds.

Claim 56. Throughout the algorithm, Property 50(a) holds.

Proof. We show this property by induction. Note that at the beginning of the algorithm, this property holds since $y(u) = W - \varepsilon$ for all $u \in L$. Since $1/\varepsilon$, and W are integers, this claim holds for all $y(u)$ with $u \in L$. Moreover, $y(u) = 0$ for all $u \in R$, and $z(e) = 0$ for all $e \in E$, thus, these are also multiples of ε. Next, look at each of the steps where the duals are modified. Consider the step Algorithm 4, if $z(e)$ was a multiple of ε before this step, then it is a multiple of ε after this step as well since $w(e)$ is an integer.

Finally, consider the Dual Adjustment procedure. If the duals are multiples of ε initially, then they are multiples of ε even after this step, since we modify the duals by ε amount. □

We now show that domination property holds.

Claim 57. Throughout the algorithm, Property 50(b) holds.

Proof. We show this property by induction. Note that initially, for all $u \in L$, $y(u) = W - \varepsilon$, so domination condition holds for all edges initially. Now consider the step Algorithm 4, after this step, $yz(e) \geqslant w(e) - \varepsilon$, which satisfies domination for e, and since we only modified $z(e)$, this doesn't affect any other edges.

Finally, consider the dual adjustment procedure. Consider e_f between $L \cap Z$ and $R \setminus Z$. Note that e_f has to be ineligible, thus $yz(e) > w(e) - \varepsilon$, and due to granularity, we can deduce that $yz(e) \geqslant w(e)$. Thus, the dual adjustment step preserves domination for such edges. Next, consider a backward edge e_b between $L \cap Z$ and $R \setminus Z$. If $yz(e) \geqslant w(e)$, then dual adjustment step preserves domination. On the other hand, if $yz(e) = w(e) - \varepsilon$, then we increment $z(e)$ as well, therefore domination is still preserved. Consider edges between $R \cap Z$ and $L \setminus Z$, since duals of $R \cap Z$ are incremented, and the duals of $L \setminus Z$ are unchanged, the domination property is preserved for such edges. Edges between $L \cap Z$ and $R \cap Z$ are not affected. □

We now show the tightness property.

Claim 58. Throughout the algorithm, Property 50(c) property holds.

Proof. We prove this by induction. At the start of the algorithm, there are no backward edges, so tightness property holds trivially. Consider Algorithm 4, this step only reduces the dual, so it can't affect the tightness property. Consider the dual adjustment procedure. Here, Algorithm 4 can affect the tightness property. However, this line only applies to backward edges e_b which have $yz(e) = w(e) - \varepsilon$, thus after dual adjustment $yz(e) = w(e)$. Algorithm 4 can affect backward edges between $L \setminus Z$ and $R \cap Z$. However, all such edges e_b must be ineligible. Thus, $yz(e) < w(e) + \varepsilon$, which by granularity implies that $yz(e) \leqslant w(e)$. Thus, after the dual adjustment step, $yz(e) = w(e) + \varepsilon$. □

We now show the free duals property.

Observation 59. Throughout the algorithm, Property 50(d) holds.

Proof. This is evident from the fact that all free vertices of L are contained in Z, and duals of all these vertices is decremented by the same amount. Similarly, Claim 55 suggests that all free vertices of R are contained in $R \setminus Z$. Since, free vertex duals on the right are initially 0, and are never incremented since they are contained in $R \setminus Z$. Thus, they remain 0 throughout. □

Finally, we show complementary slackness.

Claim 60. Throughout the algorithm, Property 50(e) holds.

Proof. We will again show this by induction. This property holds at the beginning of the algorithm since $z(e) = 0$ for all $e \in E$. Observe that this property may be violated if $z(e) > 0$ and e_b is present in G_x^t, and e_b is used in some augmenting path. However, Algorithm 4 ensures this doesn't happen. On the other hand, in Algorithm 4, we are incrementing $z(e)$, and this may violate complementary slackness as well. However, we show that for such an edge $x(e) = \kappa(e)$. To see this, note that $yz(e) = w(e) - \varepsilon$, so if $x(e) < \kappa(e)$, then $e_f \in G_x^t$. But this cannot happen since we can't haven any eligible forward edges from $L \cap Z$ to $R \setminus Z$. Thus, this step doesn't contradict complementary slackness. □

Now, we focus on the runtime analysis.

4.5 Runtime

To prove the guarantees on the runtime of Algorithm 4, we start with the following structural lemma about G_x^t

Observation 61. After dual adjustment step, there are only forward edges between $L \cap Z$ and $R \setminus Z$, and backward edges between $R \cap Z$ and $L \setminus Z$.

Proof. Suppose there is a backward edge e_b between $L \cap Z$ and $R \setminus Z$ after the dual adjustment step. This implies, that after the dual adjustment step, $yz(e_b) = w(e) + \varepsilon$. On the other hand, prior to the dual adjustment step, we had that $yz(e_b) = w(e) + 2\varepsilon$. However, this would contradict the tightness property (see Property 50(c)). Similarly, suppose there is a forward edge e_f between $R \cap Z$ and $L \setminus Z$ after the dual adjustment step. Then, this would imply that $yz(e_f) = w(e) - \varepsilon$. Thus, before the dual adjustment step, $yz(e_f) = w(e) - 2\varepsilon$, which would contradict the domination property (see Property 50(b)). □

Lemma 62. The graph G_x^t that is fed into Augmentation procedure is acyclic.

Proof. We will show this by induction. Observe that at the beginning of the algorithm, there are no back edges, so the graph is acyclic. We now show that if no cycles are present in the graph, then dual adjustment and augmentation procedures cannot create cycles.

(a) First consider Algorithm 4, observe that if $e_b \in G_x^t$ is subjected to this, then $x(e) = \kappa(e)$ due to complementary slackness. Thus e_f does not exist, and cannot be added to G_x^t due to this modification. Moreover, this modification does not affect any other edge apart from e. Thus, if the graph G_x^t was acyclic before, then it remains acyclic after Algorithm 4.

(b) Let $\vec{y}$ be the fractional matching after augmentation step. As we saw in Claim 55, $E(G_y^t) \subseteq E(G_x^t)$. Thus, if G_x^t is acyclic, then G_y^t is acyclic as well.

(c) Now consider Algorithm 4. By induction hypothesis, there are no cycles in G_x^t before Algorithm 4 was executed. Suppose a cycle C exists in G_x^t after this step, then C contains either e_f or e_b, where e is an edge subjected to Algorithm 4. However, for e_b subjected to this, $\kappa(e) = x(e)$, and therefore, e_f cannot exist in G_x^t. Moreover, $e_b \notin G_x^t$ since before the dual modification $yz(e) = w(e) - \varepsilon$, and after dual adjustment, $yz(e) = w(e)$.

(d) Observe that by induction hypothesis, there are no cycles in G_x^t before Algorithm 4 and Algorithm 4. Note that these steps preserve eligibility of any edge between $L \cap Z$ and $R \cap Z$ and those between $L \setminus Z$ and $R \setminus Z$. Thus, if a new cycle is created after Algorithm 4 and Algorithm 4, then it must contain a directed edge from Z to $V \setminus Z$, and a directed edge from $V \setminus Z$ to Z. However, Observation 61 suggests that after this step, the latter edges are not in G_x^t. This shows our claim.

□

We now proceed with runtime guarantees.

Observation 63. Observe that by definition of augmenting paths, a matched vertex is never made free. Thus, by step Algorithm 4, at the end of each iteration of the while loop, the value of free duals in $L \cap Z$ drops by ε. This implies that the total number of iterations is $O(\frac{W}{\varepsilon})$.

Lemma 64. [ST83] If the residual graph is acyclic, then we can find a maximal set of augmenting paths in time $O(m \log n)$.

Lemma 65. The total runtime of the algorithm is $O(m/\varepsilon \cdot W \cdot \log n)$.

Proof. In a particular iteration of a while loop, we see how each of the steps contribute to the runtime. Consider step Algorithm 4, this takes time $O(m)$ and similarly, the subsequent step of updating G_x^t takes time $O(m)$ as well. Since G_x^t is acyclic (by Lemma 62), we can use Lemma 64 to find the maximal set of augmenting paths in time $O(m \cdot \log n)$. Finally, the dual adjustment steps can also be accomplished in time $O(m)$. From Observation 63 implies the total runtime is $O(m/\varepsilon \cdot W \cdot \log n)$. □

5 Algorithm WeightedM-or-E*()

In this section, we now put everything together and describe the algorithm WeightedM-or-E*(). First, some definitions are in order.

Definition 66. Given a matching M, define $E_L^M(G, \kappa) = E_L(G, \kappa) \cap M$, and V_L^M to be the set of vertices that are the endpoints of $E_L^M(G, \kappa)$.

Definition 67. Given a multigraph G with weight function w and capacity function κ on the edges of G, we define the bipartite double cover of G, denoted $\text{bc}(G)$, with capacity function κ_{bc} and weight function w_{bc} as follows:

(a) For every vertex $v \in V(G)$, make two copies v and v' in $V(\text{bc}(G))$.

(b) If e is an edge between u, v, then we add an edge e' between u and v' and an edge e'' between u' and v. Moreover, $w_{\text{bc}}(e') = w_{\text{bc}}(e'') = w(e)$ and $\kappa_{\text{bc}}(e') = \kappa_{\text{bc}}(e'') = \kappa(e)$.

Observation 68. For any weighted graph G, with capacity function κ, we have, $\mathsf{mwm}(\text{bc}(G), \kappa_{\text{bc}}) \geqslant 2 \cdot \mathsf{mwm}(G, \kappa)$.

Lemma 69. Given a weighted multigraph G (possibly non-bipartite) with capacity function κ, $\varepsilon \in (0, 1)$, with edge weights from $\{1, 2, \cdots, W\}$, and with the property that for all $i \in [W]$, and for all vertices $u, v \in V$ we have, $\kappa(D_i((u, v))) \leqslant 1/\alpha_\varepsilon$, then there is an algorithm Weighted-Frac-Match-General() that takes as input $G, \kappa, w, \varepsilon$ and outputs a fractional matching $\vec{x}$ such that $\sum_{e \in E} w(e) \cdot x(e) \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}(G, \kappa)$, and further $\vec{x}$ obeys the capacities and the odd set constraints.

Proof. In the previous section, we saw the algorithm Weighted-Frac-Match() that solved this problem for bipartite graphs. For the case of general graphs, we proceed as follows. We run Weighted-Frac-Match($\text{bc}(G), \kappa_{\text{bc}}, w_{\text{bc}}, \varepsilon$), and obtain a fractional matching $\vec{z}$. To translate this into a valid fractional matching $\vec{x}$ in G, do the following: for each $e \in G$, consider $e', e'' \in \text{bc}(G)$, and let $x(e) = \frac{z(e') + z(e'')}{2}$. Note that $\vec{z}$ is a fractional matching, since $\vec{x}$ is and moreover since $\vec{x}$ satisfies capacity constraints since $\vec{x}$ does. Applying Observation 68, we know that $\sum_{e \in E} w(e) \cdot x(e) \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}(G, \kappa)$. Moreover, $\frac{\vec{x}}{1+\varepsilon}$ satisfies all odd set constraints since it satisfies the premise of Observation 71. □

We now state our main algorithm.

Algorithm 5 WeightedM-or-E*$(G, \kappa, \varepsilon, \mu, w)$

1: Include each edge $e \in E(G)$ independently with probability $p(e) = \kappa(e) \cdot \rho_\varepsilon$ into graph G_s.
2: Let $M, \vec{y}, \vec{r}$ be the output of Static-Weighted-Match(G_s, ε). ▷ Phase 1.
3: if $w(M) \leqslant (1 - 6\varepsilon) \cdot \mathsf{mwm}(G)$ then
4: Return $E^* = \{e \mid yr(e) < (1 - \varepsilon) \cdot w(e)\}$. ▷ Phase 3
5: else ▷ Phase 2
6: $M_I \leftarrow M \setminus E_L(G, \kappa)(G, \kappa)$
7: $\vec{y} \leftarrow M_I^D$ ▷ Convert M_I into a matching on multigraph
8: $\vec{x} \leftarrow$ Weighted-Frac-Match-General$(G[V_L^M] \cap E_L(G, \kappa), \kappa^+, \varepsilon, w)$
9: end if
10: Return $\vec{y} + \vec{x}$.

We restate Lemma 20, then proceed to show that Algorithm 5 satisfies the conditions of the lemma.

Lemma 20. Let G be a multigraph with edge weights in $\{1, 2, \cdots, W\}$. Then, there is an algorithm WeightedM-or-E*() that takes as input $G, \kappa, \varepsilon \in (0, 0.5)$, and a parameter $\mu \geqslant \mathsf{mwm}(G) \cdot (1 - \varepsilon)$, and in time $O(m \cdot W \cdot \log n/\varepsilon)$ returns one of the following.

(a) A fractional matching $\vec{x}$ such that $\sum_{e \in E} w(e) \cdot x(e) \geqslant (1 - 2\varepsilon) \cdot \mathsf{mwm}(G)$, with the following properties.

(i) Let $e \in D_i(u, v)$ such that $e \in \mathrm{supp}(\vec{x})$, with $\kappa(D_i(u, v)) \geqslant 1/\alpha_\varepsilon^2$, $x(D_i(u, v)) = 1$, and we have, $x(e) = \frac{\kappa(e)}{\kappa(D_i(u,v))}$. Moreover, since $\vec{x}$ is a fractional matching, we have $x(D_j(u, v)) = 0$ for all $j \neq i$.

(ii) Consider any $e \in D_i(u, v)$ such that $e \in \mathrm{supp}(\vec{x})$, with $\kappa(D_i(u, v)) \leqslant 1/\alpha_\varepsilon^2$, then $x(D_i(u, v)) \leqslant \kappa(D_i(u, v)) \cdot \alpha_\varepsilon$, and $x(e) \leqslant \kappa(e) \cdot \alpha_\varepsilon$. Moreover, for any $e \in D_j(u, v)$ with $\kappa(D_j(u, v)) > 1/\alpha_\varepsilon^2$, we have $x(e) = 0$.

(b) A set of edges E^* such that $\sum_{e \in E^*} w(e) \cdot \kappa(e) = O(\mathsf{mwm}(G) \cdot \log n)$, and for any integral matching M with $w(M) \geqslant (1 - 2\varepsilon) \cdot \mathsf{mwm}(G)$, we have $w(M \cap E^*) \geqslant \varepsilon \cdot \mathsf{mwm}(G)$. Additionally, for all $e \in E^*$, we have $\kappa(e) < 1$.

Proof. We first show the runtime of the algorithm. First note that G_s can be computed in $O(m)$ time since this only involves sampling edges independently. Moreover, by Lemma 42, we know that the runtime of Static-Weighted-Match(G, ε) is $O(m/\varepsilon \cdot \log 1/\varepsilon)$. Additionally, from Lemma 44, we can conclude that we can obtain set E^* using the output of Static-Weighted-Match(G_s, ε) and the time taken to do this is $O(m)$. Finally, from Lemma 69, we can conclude that we can run Line 8 in $O(m/\varepsilon \cdot \log 1/\varepsilon \cdot W \cdot \log n)$.

Now, we turn to show Lemma 20(a). In this case, first note that V_L^M and $V(M_I)$ are disjoint. This follows from Algorithm 5 and Definition 66. Thus, since $\vec{y}, \vec{x}$ are fractional matchings, we can conclude that $\vec{z}$ is also a fractional matching. Note that we land in Lemma 20(a) if $w(M) \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}(G)$. From Lemma 41 we can conclude that for $H = E_L(G, \kappa) \cap G[V_L^M]$ and $H_s = E_L(G, \kappa) \cap G_s[V_L^M]$, we have that $\mathsf{mwm}(H_s) \leqslant \mathsf{mwm}(H, \kappa^+) + \varepsilon \cdot \mathsf{mwm}(G)$. Thus, we have,

$$\begin{aligned}
\sum_{e \in E} y(e) \cdot w(e) + \sum_{e \in E} x(e) \cdot w(e) &\geqslant w(M_I) + \mathsf{mwm}(H, \kappa^+) - \varepsilon \cdot \mathsf{mwm}(G) \\
&\geqslant w(M_I) + \mathsf{mwm}(H_s) - 2\varepsilon \cdot \mathsf{mwm}(G) \\
&= w(M) - 2\varepsilon \cdot \mathsf{mwm}(G) \\
&\geqslant (1 - \varepsilon) \cdot \mathsf{mwm}(G) - 2\varepsilon \cdot \mathsf{mwm}(G)
\end{aligned}$$

We now proceed to proof Lemma 20(a)(i) and (ii). Consider any edge $e \in \mathrm{supp}(\vec{z})$ with $w(e) = i$ and $\kappa(D_i(e)) \leqslant 1/\alpha_\varepsilon^2$. Thus, we know that $e \in \mathrm{supp}(\vec{x})$, and so by Lemma 69, we have that $z(e) = x(e) \leqslant \kappa^+(e) \leqslant$

$\kappa(e) \cdot \alpha_\varepsilon$ and $z(D_i(e)) = x(D_i(e)) \leqslant \kappa^+(D_i(e)) \leqslant \kappa(D_i(e)) \cdot \alpha_\varepsilon$. Similarly, for any $e \in \mathrm{supp}(\vec{z})$ with $w(e) = i$, and $\kappa(D_i(e)) > 1/\alpha_\varepsilon^2$. We know that $e \in \mathsf{supp}(\vec{y})$. By definition of $\vec{y}$, we have $z(e) = y(e) = \kappa(e)/\kappa(D_i(e))$, and $z(D_i(e)) = 1$, this proves our claim.

Finally, we prove Lemma 20(b). First consider Lemma 44, this lemma implies that $\sum_{e \in E^*} \kappa(e) \cdot w(e) = O(\mathsf{mwm}(G) \cdot \log n)$. Finally, by Claim 43, we have, $\sum_{e \in M \cap E^*} w(e) \geqslant w(M) - (1+\varepsilon)^2 \cdot \mathsf{mwm}(G_s)$, thus, for any M with $w(M) \geqslant (1-\varepsilon) \cdot \mathsf{mwm}(G)$, we have $\sum_{e \in M \cap E^*} w(e) \geqslant \varepsilon \cdot \mathsf{mwm}(G)$. This is implied by the fact that the algorithm returns E^* when $\mathsf{mwm}(G_s) \leqslant (1-5\varepsilon) \cdot \mathsf{mwm}(G)$. Finally, for all edges in E^*, $\kappa(e) < 1$, otherwise they'd be sampled into G_s. □

Acknowledgement

Thank you to Jiale Chen, Aaron Sidford, and Ta-Wei Tu for coordinating posting to arXiv.

This work is supported by Austrian Science Fund (FWF): P 32863-N. This project has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (grant agreement No 947702).

A Missing Proofs

We start with the following observation:

Observation 70. Suppose $\vec{x}$ is a fractional matching that satisfies odd set constraints for all odd sets of size smaller than $3/\varepsilon + 1$, then the fractional matching $\vec{z} = \frac{\vec{x}}{(1+\varepsilon)}$ satisfies odd set constraints for all odd sets.

Proof. Suppose $\vec{x}$ is a fractional matching that satisfies odd set constraints for all odd sets of size at most $3/\varepsilon$. Let $\vec{z} = \frac{\vec{x}}{1+\varepsilon}$. Now consider any odd set B such that $|B| \geqslant 3/\varepsilon + 1$. Then, we have,

$$\frac{|B|}{|B|-1} = 1 + \frac{1}{|B|-1} \leqslant 1 + \varepsilon/3. \tag{6}$$

Now since $\vec{x}$ satisfies fractional matching constraints, we have, $\sum_{e \in G[B]} x(e) \leqslant |B|/2$. Thus, we have, using Equation (6):

$$\sum_{e \in G[B]} z(e) \leqslant \sum_{e \in G[B]} \frac{x(e)}{1+\varepsilon} \leqslant \frac{|B|}{2 \cdot (1+\varepsilon)} \leqslant \frac{|B|-1}{2}$$

This proves the observation. □

Observation 71. Let $\vec{f}$ be a fractional matching on a multigraph, that puts flow at most ε between every pair of vertices $u, v \in V$. Then, $\vec{f}$ satisfies odd set constraints of sets of size at most $1/\varepsilon$. Thus, $\vec{f}$ satisfies all odd-set constraints.

Proof. Consider any odd set B such that $|B| \leqslant 1/\varepsilon$, then,

$$\sum_{e \in G[B]} f(e) \leqslant \sum_{u,v \in B} f((u,v)) \leqslant \varepsilon \cdot \frac{|B| \cdot (|B|-1)}{2} \leqslant \frac{|B|-1}{2}.$$

From Observation 70 we can conclude that $\frac{\vec{f}}{1+\varepsilon}$ satisfies all odd set constraints. □

B Reduction To Multigraphs with Heavy Matchings

In this section, we prove the following theorem.

Lemma 10. Let $\delta \in (0, 0.5)$ and G be a weighted graph with weights in $\{1, 2, \cdots, W\}$. Let $\mathsf{mwm}(G) \geqslant \mu$ and at most $(1+\delta) \cdot \mu$. There is an algorithm Vertex-Red(G, μ, δ) that in time $O(m \cdot W^3 \cdot \log n / \delta^4)$ returns $\lambda = \frac{100 \cdot \log n \cdot W^3}{\delta^4}$ multigraphs $H_1, \cdots, H_\lambda$ that have the following properties, where the second property holds with probability at least $1 - 1/n^2$.

(a) For all $i \in [\lambda]$, $|V(H_i)| = \frac{4 \cdot (1+\delta) \cdot W \cdot \mathsf{mwm}(G)}{\delta}$, $E(H_i) \subseteq E(G)$, and

(b) Suppose G is subject to deletions, which we simulate on each of the H_i's. More concretely, if an edge e is deleted from G, then its corresponding copy in H_i is also deleted. Suppose at the end of the deletion process, $\mathsf{mwm}(G) \geqslant \tilde{\mu}$ for some $\tilde{\mu} \geqslant (1-2\delta) \cdot \mu$, then $\mathsf{mwm}(H_i) \geqslant (1-\delta) \cdot \tilde{\mu}$ for some $i \in [\lambda]$.

Observation 72. Let G be a simple graph, then it contains at most $2^{O(k \cdot \log n)}$ matchings which have k edges.

Proof. This is evident from the fact that there are at most $\binom{n^2}{k}$ ways of choosing k edges from among n^2 edges in total. □

Now, in order prove Lemma 10, we give a simple procedure that takes as input a simple graph, and outputs a multigraph on $O(\mathsf{mwm}(G) \cdot W/\varepsilon)$ vertices that preserves the weight of a fixed matching M of G with probability at least $1 - \exp\left(-O(|V(M)| \cdot \varepsilon^3 / W^3)\right)$.

Algorithm 6 Vertex-Red-Basic(G, τ, ε)

Input: Graph G, a sparsification parameter τ, and a parameter $\varepsilon > 0$
Output: A multigraph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ with $|\mathcal{V}| = \tau$

1: Partition V into τ bins, $\mathcal{V} \coloneqq (B_1, B_2 \cdots, B_\tau)$, by assigning every vertex to one of the τ bins uniformly at random.
2: For a vertex u, let $B(u)$ denote the bin chosen for u. For any edge $(u, v) \in E$, if $B(u) \neq B(v)$, then add an edge $e_{u,v}$ between $B(u)$ and $B(v)$.
3: Return the multigraph $\mathcal{G}(\mathcal{V}, \mathcal{E})$.

Lemma 73. Let G be a simple weighted graph with weights in $\{1, \cdots, W\}$, and let $\tau = \frac{8 \cdot (1+\delta) \cdot W \cdot \mathsf{mwm}(G)}{\varepsilon}$. Let $\mathcal{G} = $ Vertex-Red-Basic(G, τ, ε), and let M be any fixed matching of G. Then, there exists a matching $\mathcal{M} \subset M$ in $\mathcal{G}$ such that $w(\mathcal{M}) \geqslant (1-\varepsilon) \cdot w(M)$ with probability at least $1 - 2^{-\frac{|V(M)| \cdot \varepsilon^3}{32 \cdot W^3}}$.

Proof. Let $\delta = \varepsilon/4$, and let $t = 2 \cdot W \cdot |V(M)|$. Note that by our definition, $\delta < 1/2$. Recall that there are $\tau = \frac{8 \cdot (1+\delta) \cdot W \cdot \mathsf{mwm}(G)}{\varepsilon}$ bins, we combine these arbitrarily so that there are now t/δ groups. Call these groups $Z_1, \cdots, Z_{t/\delta}$. Note that every vertex v lands in Z_i with probability δ/t. We call a group bad if it doesn't contain even one of the $2 \cdot |V(M)|$ vertices of M. Let X_i be the random variable that takes value 1 if Z_i is bad, and is 0 otherwise. So, we have,

$$\begin{aligned}
\Pr(X_i = 1) &= \left(1 - \frac{\delta}{t}\right)^{2 \cdot |V(M)|} \\
&\leqslant e^{-\delta/W} \\
&\leqslant 1 - \delta/W + \delta^2/2 \cdot W^2
\end{aligned}$$

Let X be a random variable denoting the number of bad groups, that is, $X = \sum_{i=1}^{t/\delta} X_i$. Note that we have,

$$\mathbb{E}[X] \leqslant t/\delta (1 - \delta/W + \delta^2/2 \cdot W^2).$$

Note that X is a sum of negatively correlated random variables. Thus, using Chernoff bound, we have,

$$\begin{aligned}
\Pr\left(X \geqslant (1 + (\delta/W)^2) \cdot t/\delta \cdot (1 - \delta/W + \delta^2/2 \cdot W^2)\right) &\leqslant \exp\left(-(\delta/W)^4 \cdot t/\delta \cdot (1 - \delta/W + \delta^2/2 \cdot W^2)\right) \\
&= \exp\left(-\frac{\delta^3 \cdot t}{W^4} + \frac{\delta^4 \cdot t}{W^5}\right)
\end{aligned}$$

$$\leqslant \exp\left(-\frac{\delta^3 \cdot t}{2 \cdot W^4}\right)$$

Thus, with probability at least $1 - \exp\left(-\frac{\delta^3 \cdot t}{2 \cdot W^4}\right)$, we have, $X \leqslant t/\delta \cdot (1 - \delta/W + 2\delta^2/W^2)$. This implies that the number of good groups is at least $2|V(M)| - 4 \cdot \delta/W \cdot |V(M)|$. Now, we want to show there is a matching $\mathcal{M}$ among these groups such that $w(\mathcal{M}) \geqslant (1 - 4 \cdot \delta) \cdot w(M)$. To see this, in every group, we fix one vertex of M, and remove the rest. This way, we deleted at most $4 \cdot \delta/W \cdot |V(M)|$ edges of M, and these can contribute total weight at most $4 \cdot \delta \cdot |V(M)| \leqslant 4 \cdot \delta \cdot w(M)$ to M. Thus, we have in the remaining matching, which we call $\mathcal{M}$, $w(\mathcal{M}) \geqslant (1 - 4 \cdot \delta) \cdot w(M)$. □

Lemma 74. Let Vertex-Red() be an algorithm that does independent runs of Vertex-Red-Basic(). Let $H_1, \cdots, H_\lambda$ be multigraphs obtained on independent runs of Vertex-Red-Basic() on input a weighted graph G with weights in $\{1, \cdots, W\}$, $\tau \geqslant \frac{4 \cdot \mathrm{mwm}(G) \cdot W}{\varepsilon}$ and $\varepsilon \in (0, 1/2)$, for $\lambda \geqslant \frac{1000 \log n \cdot W^3}{\varepsilon^4 \cdot (1-\varepsilon)}$. Then, with probability at least $1 - O(1/n^2)$, for all matchings M of G, there is a matching M' in some H_i such that $M' \subset M$ and $w(M') \geqslant (1 - \varepsilon) \cdot w(M)$.

Proof. Consider a fixed matching M, with $|M| = k$. Then, with probability at least $1 - \exp\left(-\frac{\varepsilon^3 \cdot k}{32 \cdot W^3}\right)$, a fixed H_i contains $M' \subset M$, with $w(M') \geqslant (1 - \varepsilon) \cdot w(M)$. Since each of the H_j are independent runs of Vertex-Red-Basic(), with probability at least $1 - \exp\left(-\frac{k \cdot \log n}{\varepsilon}\right)$, some H_i contains a matching M' such that $M' \subset M$, and $w(M') \geqslant (1 - \varepsilon) \cdot w(M)$. Taking a union bound over all possible matchings with k edges, and then over all possible values of k, we have the claim we need. □

We now show the following.

Lemma 75. Let G be a simple weighted graph, with weight function w on the edges, and weights in $\{1, 2, \cdots, W\}$. Let $H_1, \cdots, H_\lambda$ be output of independent runs of Vertex-Red-Basic$(G, \lambda, \varepsilon)$, where $\lambda \geqslant \frac{1600 \cdot \log(n \cdot W)}{\varepsilon^4 \cdot (1-\varepsilon)}$. Consider an adversary that deletes edges from G. Suppose we simulate these deletions on each of the H_i. That is, if (u, v) is deleted from G, then $e_{u,v}$ (if present) is deleted from each of the H_i. Let mwm_i^t denote the weight of $\mathsf{mwm}(H_i)$ at time t, and let mwm^t denote the weight of $\mathsf{mwm}(G)$ at time t. Suppose $\mathsf{mwm}^t \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}^0$, then $\mathsf{mwm}_i^t \geqslant (1 - 2\varepsilon) \cdot \mathsf{mwm}^0$ for some $i \in [\lambda]$.

Proof. Let M be a matching satisfying the assumption of the lemma. That is, let M be a matching at time t in G such that $w(M) \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}^0$. Since the adversary is only performing deletions, thus we have, initially, before all deletions are performed, $w(M) \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}^0$. Thus, by Lemma 74, we know that there is an H_i containing $M' \subset M$ with the property that $w(M') \geqslant (1 - \varepsilon) \cdot w(M)$. Since all edges of M survive at time t, this is true for M' as well. Consequently, we have that at time t, $w(M') \geqslant (1 - \varepsilon)^2 \cdot \mathsf{mwm}^0$. This proves our claim. □

Proof of Lemma 10. Lemma 10(a) follows from the properties of Vertex-Red(), Lemma 73, and Lemma 74. On the other hand, Lemma 10(b) follows from the contrapositive of Lemma 75. □

C Properties of Static-Weighted-Match

In this section, we show Lemma 42, which we first restate.

Lemma 42. [DP14] There is an $O(m/\varepsilon \cdot \log 1/\varepsilon)$ time algorithm Static-Weighted-Match() that takes as input, a weighted graph G, and a parameter $\varepsilon > 0$, and outputs an integral matching M, and dual vectors $\vec{y}$ and $\vec{r}$ with the following properties.

(a) It returns an integral matching M such that $w(M) \geqslant (1 - \varepsilon) \cdot \mathsf{mwm}(G)$

(b) A set Ω of laminar odd-sized sets such that $\{B \in V_{\text{odd}} \mid r(B) > 0\} \subseteq \Omega$.

(c) For all odd-sized sets B such that $|B| \geqslant 1/\varepsilon + 1$, $r(B) = 0$.

(d) For all $v \in V$, $y(v)$ is an integral multiple of ε, and for all B with $|B|$ odd, $r(B)$ is an integral multiple of ε.

(e) For each edge $e \in E$, we have $yr(e) \geqslant (1-\varepsilon) \cdot w(e)$, that is e is approximately covered by $\vec{y}$ and $\vec{r}$.

(f) The value of the dual objective, $f(y,r)$ is at most $(1+\varepsilon) \cdot \mathsf{mwm}(G)$.

We first note that the properties (a)-(b) and (d)-(f) are evident in Property 3.1 and the proof of Lemma 2.3 of [DP14]. We now show that (c) also holds.

Proof of Lemma 42. Let Basic-Static-Weighted-Match() be the algorithm satisfying of [DP14] satisfying (a)-(b) and (d)-(f). We now show how to obtain Static-Weighted-Match() from Basic-Static-Weighted-Match(). We run Basic-Static-Weighted-Match() with input G and $\delta = \varepsilon/3$. Thus, we get a matching M with $w(M) \geqslant (1-\varepsilon/3) \cdot \mathsf{mwm}(G)$. Moreover, the duals $\vec{y}$ and $\vec{z}$ output by the algorithm have $f(y,r) \leqslant (1+\varepsilon/3) \cdot \mathsf{mwm}(G)$. We consider the following procedure: for every $B \in \Omega$ with $z(B) > 0$ and $|B| \geqslant \frac{3}{\varepsilon} + 1$, we increase $y(v)$ by $\frac{z(B)}{2}$ for every $v \in B$, and we decrease $z(B)$ to 0. Thus, each $y(v)$ is still a multiple of ε and each $z(B)$ is still a multiple of ε. We conclude that (d) still holds. Moreover, this transformation keeps the value of the dual constraint for every edge the same. Thus, we still satisfy (e). We update Ω by removing B from it. The set Ω still remains laminar. Thus, (b) is still satisfied. We first observe that the time taken to do this is linear in the sum of the sizes of the odd sets B with $z(B) > 0$. Note that since $yz(V)$ sums to at most $(1+\varepsilon/3)\mathsf{mwm}(G)$, and non-zero $z(B)$ have value at least $\varepsilon/3$, this implies that the sums of the sizes of the odd sets is at most $3/\varepsilon \cdot (1+\varepsilon/3) \cdot \mathsf{mwm}(G)$. Thus, the time taken for the procedure is $O(m/\varepsilon)$. Next, we observe that the value of $yz(V)$ changes by at most $\sum_{B:|B| \geqslant 3/\varepsilon+1} \frac{z(B)}{2}$. This value is at most $\varepsilon/3 \cdot (1+\varepsilon/3) \cdot \mathsf{mwm}(G)$, as shown by the following calculation.

$$\left(\frac{3}{\varepsilon}\right) \cdot \sum_{B:|B| \geqslant 3/\varepsilon+1} \frac{z(B)}{2} \leqslant \sum_{B:|B| \geqslant 3/\varepsilon+1} \left(\frac{|B|-1}{2}\right) \cdot z(B) \leqslant (1+\varepsilon/3) \cdot \mathsf{mwm}(G)$$

Thus, the new $yz(V)$ has value at most $(1+\varepsilon/3)^2 \cdot \mathsf{mwm}(G)$. This implies that (f) holds. Finally, since every blossom B of size at least $3/\varepsilon + 1$ has $z(B) = 0$, thus, (c) holds. □

D Matchings in Decremental Graphs with $\mathsf{mwm}(G) = O(\log n)$

As mentioned in the introduction, the underlying assumption in our paper is that the decremental graph G has $\mathsf{mwm}(G) = \Omega(\log n)$ as mentioned in Assumption 13. We now give a simple decremental algorithm for the case where the graph G has $\mathsf{mwm}(G) = O(\log n)$. We start by recalling Lemma 9, which allows us to focus on designing an algorithm for the case where the weights are integers in $\{1, \cdots, W\}$. Using this assumption, we have the following observation.

Observation 76. If G is an integer-weighted graph with weight function w on the edges, with weights in $\{1, 2, \cdots, W\}$. Then, $\mathsf{mwm}(G) \geqslant \mathsf{mcm}(G)$. Consequently, if $\mathsf{mwm}(G) = O(\log n)$, then there is a vertex cover of G of size $O(\log n)$.

Given a graph G, we give a weighted matching sparsifier for G. This sparsifier is mentioned in [GP13] for unweighted graphs, but the proof extends to weighted graphs as well, and we state it here for completeness.

Definition 77. Given a graph G, and a vertex cover S, the core graph H consists of the following edges:

(a) All edges in $G[S]$.

(b) For each vertex $v \in S$, $|S|+1$ heaviest edges from v to $V \setminus S$.

Lemma 78. [GP13] Given a graph G, and a core graph H of G, we have $\mathsf{mwm}(H) = \mathsf{mwm}(G)$.

Proof. Suppose $\mathsf{mwm}(G) > \mathsf{mwm}(H)$. Let M^* denote the optimal matching in G that uses the most number of edges from H. There is an edge $e \in M^* \setminus H$. Observe that since S is a vertex cover, and we have $G[S] \subseteq H$, this implies that e has one end point in $u \in S$ and the other in $v \in V \setminus S$. Note that e wasn't included into H since there u has at least $|S|+1$ edges incident on it in H, and for each of these edges e', we have $w(e') \geqslant w(e)$. Next, observe that one of these neighbours of u in $V \setminus S$ must be unmatched (call this w). This is because S is a vertex cover, and there are no edges among vertices in $V \setminus S$, and consequently, at most $|S|$ vertices in $V \setminus S$ can be matched. Substituting (u, v) with (u, w) in M^*, we get a matching M' in G which uses even more edges from H and $w(M') \geqslant w(M^*)$. This is a contradiction. □

Lemma 79. Consider a decremental graph G with $\mathsf{mwm}(G) = O(\log n)$. Then, there is a decremental algorithm for maintaining a $(1-\varepsilon) - \mathsf{mwm}(G)$ in $O(\log^2 n)$ update time.

Proof. Given a decremental graph G with $\mathsf{mwm}(G) = O(\log n)$. We can in $O(m)$ time preprocess the graph to sort the edges of every vertex and store these as n doubly linked lists. We can in $O(m)$ time compute a maximal matching of G, and using this in compute a vertex cover S of G with $|S| = O(\log n)$. Recall, the G is decremental, therefore, S always remains a vertex cover of G. Thus, we never need to recompute S. Next, we a compute the core graph H in $O(\log^2 n)$ time. Using a standard approximate weighted matching algorithm, we can compute a $(1-\varepsilon)$ approximate matching M of H in time $O(\log^2 n)$ (see Lemma 11). As we process deletions, if an edge is deleted from $G \setminus H$, we just remove that edge from the list and do nothing. If an edge is deleted from H, then in addition to doing the above-mentioned step, we also delete the edge from H. If this was an edge from a vertex $u \in S$ to a vertex $v \in V \setminus S$, then we add to H, the next highest weighted edge from u to $V \setminus S$ and update the doubly-linked list in $O(1)$ time. Finally, if this edge was also in M, we just recompute a new matching of H from scratch in time $O(\log^2 n)$ since $|E(H)| = O(\log^2 n)$. □

E Rounding Weighted Fractional Matchings

In this section, we will prove Lemma 26. More concretely, we give state the procedure Sparsification() that takes as input a fractional matching $\vec{x}$ of a simple graph G, and outputs a graph H, of size $\tilde{O}(\mu(G))$. If $x(e) \leqslant \varepsilon^6$ for all $e \in E(G)$, then, H contains an integral matching of weight at least $(1-10\varepsilon) \cdot \sum_{e \in E} w(e) \cdot x(e)$ in its support. The proof of this theorem is implicit in the work of [Waj20], but we describe it here for completeness. We first state the algorithm, and then describe some of its properties. Note that even though we work over a multigraph, and compute a fractional matching $\vec{x}$, we feed $\vec{x}^C$ into the rounding algorithm. Fractional matching $\vec{x}^C$ has the property that between every pair of vertices $u, v \in V$, there is at most one edge of a given weight. Thus, the total number of edges is at most $n^2 \cdot W$.

The algorithm uses a dynamic edge coloring algorithm as a subroutine. The update time of the subroutine is $O(\log n)$ in the worst case. The sparsification algorithm takes as input a fractional matching $\vec{x}$ and a parameter $\varepsilon > 0$. Then, it classifies $E(G)$ into classes as follows: $E_i = \left\{e \mid x(e) \in [(1+\varepsilon)^{-i}, (1+\varepsilon)^{-i+1})\right\}$. Additionally, we can limit the number of edge classes into $O(\log nW)$ many using the following argument.

Observation 80. We can assume that $\frac{\min_{e \in E} x(e)}{\max_{e \in E} x(e)} \geqslant \frac{\varepsilon^2}{n^2 \cdot W}$.

Proof. Let $F = \left\{e \in E \mid x(e) \leqslant \frac{\varepsilon^2}{n^2 \cdot W} \cdot \max_{e' \in E} x(e')\right\}$. Note that $\sum_{e \in F} w(e) \cdot x(e) \leqslant \varepsilon^2 \cdot \max_{e' \in e} x(e')$. Thus, we can throw away these edges without hurting the weight of the fractional matching by more than an $1-\varepsilon$ factor. □

From the above observation, we can conclude that the total number of edge classes are at most $\log(n \cdot W^2/\varepsilon)$ many.

We state the guarantees of the edge coloring subroutine.

Observation 81. The size of K output by Sparsification$(\vec{x}, \varepsilon)$ is,

$$|E(H)| = O\left(\frac{\log(n \cdot W/\varepsilon)}{\varepsilon} \cdot d \cdot \mu(\mathrm{supp}(H))\right)$$

Algorithm 7 Sparsification($\vec{x}, \varepsilon$)

$d \leftarrow \frac{4 \cdot \log(2/\varepsilon)}{\varepsilon^2}$
for $i \in \left\{1, 2, \cdots, 2\log_{1+\varepsilon}(n \cdot W/\varepsilon)\right\}$ do
    Compute a $3\lceil(1+\varepsilon)^i\rceil$-edge colouring Φ_i of E_i.
    Let S_i be a sample of $3 \cdot \min\left\{\lceil d\rceil, \lceil(1+\varepsilon)^i\rceil\right\}$ colours without replacement in Φ_i.
    Return $K = (V, \cup_i \cup_{M \in S_i} M)$
end for

Lemma 82. [BCHN18] There is a deterministic dynamic algorithm that maintains a $2\Delta - 1$ edge coloring of a graph G in $O(\log n)$ worst case update time, where Δ is the maximum degree of the graph G.

Observation 83. Let $\varepsilon \in (0, 1/2)$ and suppose the input to Sparsification($\vec{x}, \varepsilon$) is a matching $\vec{x}$ with $x(e) \leqslant \varepsilon^6$, then, $|S_i| = 3 \cdot \lceil d\rceil$.

To show that K contains a matching of weight at least $(1-\varepsilon) \cdot \sum_{e \in E(G)} w(e) \cdot x(e)$ in its support, we will show a random fractional matching $\vec{y}$ in K that sends flow at most ε through each of its edges. Moreover, $\mathbb{E}\left[\sum_{e \in E} w(e) \cdot y(e)\right] \geqslant (1 - 6\varepsilon) \cdot \sum_{e \in E(G)} w(e) \cdot x(e)$. This will show the existence of a large matching that sends flow at most ε through each of its edges. To show this, we give some properties of the algorithm.

Lemma 84. Let $\varepsilon \in (0, 1/2)$ and let $\vec{x}$ be the input to Sparsification($\vec{x}, \varepsilon$) such that $x(e) \leqslant \varepsilon^6$ for all $e \in E(G)$. Then, for every edge e, $\Pr(e \in K) \in [x(e) \cdot d/(1+\varepsilon)^2, x(e) \cdot d \cdot (1+\varepsilon)]$.

Proof. Since $\vec{x}$ has the property that for every $e \in E$, $x(e) \leqslant \varepsilon^6$, this implies that we sample $3 \cdot \lceil d\rceil$ from Φ_i colors for each i (from Observation 83). Thus, if we consider an edge $e \in E_i$, then, we have,

$$\Pr(e \in K) = \frac{\lceil d\rceil}{\lceil(1+\varepsilon)^i\rceil} \leqslant \frac{d \cdot (1+\varepsilon)}{(1+\varepsilon)^i} \leqslant d \cdot (1+\varepsilon) \cdot x(e)$$

The last inequality follows from the fact that $x(e) \in \left[(1+\varepsilon)^{-i}, (1+\varepsilon)^{-i+1}\right]$. Moreover, we have,

$$\frac{\lceil d\rceil}{\lceil(1+\varepsilon)^i\rceil} \geqslant \frac{d}{(1+\varepsilon)^i + 1} \geqslant \frac{d}{(1+\varepsilon)^{i+1}} \geqslant \frac{d \cdot x(e)}{(1+\varepsilon)^2}$$

The first inequality follows from the fact that $\lceil d\rceil \geqslant d$, and $\lceil(1+\varepsilon)^i\rceil \leqslant (1+\varepsilon)^i + 1$. The second inequality follows from the following reasoning.

$$(1+\varepsilon)^{-i} \leqslant x(e) \leqslant \varepsilon^6 \leqslant 1/d \leqslant \varepsilon$$

Thus, $1/\varepsilon \leqslant (1+\varepsilon)^i$, and $1 \leqslant \varepsilon \cdot (1+\varepsilon)^i$, so the second inequality follows. The last inequality follows from the fact that $x(e) \in \left[(1+\varepsilon)^{-i}, (1+\varepsilon)^{-i+1}\right]$. □

For an edge $e \in E(G)$, we define X_e to be an indicator random variable that takes value 1 if $e \in K$, and 0 otherwise.

Observation 85. For a vertex v, the variables $\{X_e \mid e \text{ incident on } v\}$ are negatively associated.

Proof. If the edges e and e' incident on v belong in E_i and E_j where $i \neq j$, then X_e and $X_{e'}$ are independent. On the other hand, if they belong in the same E_i, then recall we did an edge colouring on E_i, so e' and e got different colours. So, if the colour corresponding to e is picked into K, then this reduces the probability of the colour corresponding to e' being picked into K. □

Lemma 86. From Observation 85 and Lemma 84, we can conclude that for edges e and e' incident on v,

$$\Pr(X_e = 1 \mid X_{e'} = 1) \leqslant \Pr(X_e = 1) \leqslant x(e) \cdot d \cdot (1+\varepsilon)$$

Lemma 87. For any vertex v and edge e' incident on v, the variables $\{[X_e \mid X_{e'}] \mid e \text{ incident on } v\}$ are negatively associated.

Following theorem directly implies Lemma 26.

Theorem 88. Let K be the subgraph of G output by Sparsification(), when run on the matching $\vec{x}$ with parameters $\varepsilon \in (0, 1/2)$. If $x(e) \leqslant \varepsilon^6$ for all $e \in E$, then K supports a fractional matching $\vec{y}$ such that $y(e) \leqslant \varepsilon$, and $\sum_{e\in E} w(e) \cdot y(e) \geqslant (1 - 6\varepsilon) \cdot \sum_{e\in E} w(e) \cdot x(e)$.

Proof. To show the theorem, we will describe a process that simulates Sparsification$(\vec{x}, \varepsilon)$, and outputs a random matching $\vec{y}$ such that $y(e) \leqslant \varepsilon$ for every $e \in E$. Additionally, $\mathbb{E}\left[\sum_{e\in E} w(e) \cdot y(e)\right] \geqslant (1 - 6\varepsilon) \sum_{e\in E} w(e) \cdot x(e)$. As an intermediate step, let $z(e) = \frac{(1-4\varepsilon)}{d} \cdot X_e$. So, we have,

$$\mathbb{E}[z(e)] \geqslant \mathbb{E}[z(e) \mid X_e = 1] \cdot \Pr(X_e = 1) \geqslant \frac{1 - 4\varepsilon}{d} \cdot \frac{x(e) \cdot d}{(1 + \varepsilon)^2} \geqslant (1 - 6\varepsilon) \cdot x(e)$$

The second to last inequality follows from Lemma 84. Next define $\vec{y}$ as follows:

$$y(e) = \begin{cases} 0 \text{ if for one of the endpoints } v \text{ of } e,\ \sum_{e'\in v} z(e') > 1 \\ z(e) \text{ otherwise.} \end{cases}$$

Essentially, our procedure is creating matching $\vec{z}$ as follows: it assigns value $\frac{1-4\varepsilon}{d}$ to $z(e)$ if e was included in K, and 0 otherwise. The value $y(e)$ is the same $z(e)$ in all cases except when e is picked into K, but at one of the endpoints v, $\sum_{e'\ni v} z(e') > 1$, that is, the fractional matching constraint is violated at v. We show that it is unlikely that an edge (when picked) has one of its endpoints violated. Let e' be an edge incident on v, we to bound the probability that $z(e') \neq y(e')$.

$$\begin{aligned}
\mathbb{E}\left[\sum_{e\in v} z(e) \mid X_{e'} = 1\right] &\leqslant \frac{1 - 4\varepsilon}{d} + \mathbb{E}\left[\sum_{e'\neq e, e\in v} z(e) \mid X_{e'}\right] \\
&\leqslant \varepsilon + \sum_{e\in v} x(e) \cdot (1 + \varepsilon) \cdot d \cdot \left(\frac{1 - 4\varepsilon}{d}\right) \\
&\text{(Since } 1/d \leqslant \varepsilon \text{ and from Lemma 86)} \\
&\leqslant (1 - \varepsilon)
\end{aligned}$$

Note that at any end point of e', conditioned on e' being sampled, the expected sum of $z(e)$'s at that endpoint is upper bounded by $(1 - \varepsilon)$. Now, $z(e)$ is assigned value 0 only if the sum of the $z(e)$'s deviates from the expected value by ε. To see this, we want to compute $\mathrm{Var}[[z(e) \mid X_{e'}]]$, where e and e' share an end point. Note that $[z(e) \mid X_{e'}]$ takes value $\frac{1-4\varepsilon}{d}$ with probability $\Pr(X_e \mid X_{e'})$, otherwise it takes value 0.

$$\begin{aligned}
\mathrm{Var}[[z(e) \mid X_{e'}]] &\leqslant \mathbb{E}\left[[z(e) \mid X_{e'}]^2\right] \\
&\leqslant \left(\frac{1 - 4\varepsilon}{d}\right)^2 \cdot \Pr(X_e \mid X_{e'}) \\
&\leqslant \left(\frac{1 - 4\varepsilon}{d}\right)^2 \cdot x(e) \cdot d \cdot (1 + \varepsilon) \\
&\text{(From Lemma 86)} \\
&\leqslant \frac{x(e)}{d}
\end{aligned}$$

This implies that $\sum_{e\in v} \mathrm{Var}[[z(e) \mid X_{e'}]] \leqslant \frac{1}{d}$. So, we want to compute the probability that the sum of the random variables $\{[z(e) \mid X_{e'}]\}$ deviates from the expected value by ε. Applying Lemma 37, we have,

$$\begin{aligned}
\Pr\left(\sum_{e\in v} [z(e) \mid X_{e'}] \geqslant \mathbb{E}\left[\sum_{e\in v} [z(e) \mid X_{e'}]\right] + \varepsilon\right) &\leqslant \exp\left(\frac{-\varepsilon^2}{2\left(\frac{1}{d} + \frac{\varepsilon}{3\cdot d}\right)}\right) \\
&\text{(Since } z(e) \in [0, 1-4\varepsilon/d]) \\
&\leqslant \exp\left(-\varepsilon^2 \cdot 0.25 \cdot d\right)
\end{aligned}$$

$$
\begin{aligned}
&(\text{Since } \varepsilon \in (0,1)) \\
&\leqslant \frac{\varepsilon}{2} \\
&(\text{Since } d = \frac{4 \cdot \log(2/\varepsilon)}{\varepsilon^2})
\end{aligned}
$$

Taking union bound over both endpoints, we know that $\Pr\left(y(e) = z(e) \mid X_e = 1\right) \geqslant (1-\varepsilon)$. Thus, we have:

$$
\begin{aligned}
\mathbb{E}\left[y(e)\right] &= \left(\frac{1-4\varepsilon}{d}\right) \cdot \Pr\left(y(e) = z(e)\right) \\
&= \left(\frac{1-4\varepsilon}{d}\right) \cdot \Pr\left(y(e) = z(e) \mid X_e = 1\right) \Pr\left(X_e = 1\right) \\
&\geqslant \left(\frac{1-4\varepsilon}{d}\right) \cdot (1-\varepsilon) \cdot \frac{x(e) \cdot d}{(1+\varepsilon)^2} \\
&\geqslant (1-7\varepsilon) \cdot x(e)
\end{aligned}
$$

Thus, $\mathbb{E}\left[\sum_{e \in E} w(e) \cdot y(e)\right] \geqslant (1-7\varepsilon) \cdot \sum_{e \in E} w(e) \cdot x(e)$. Moreover, for all $e \in E$, $y(e) \leqslant \varepsilon^6$. This proves our claim. □

We now restate the lemma, and then show its proof.

Lemma 26. ([Waj20]) Suppose G is a integer-weighted multigraph, let $\varepsilon \in (0, 1/2)$, and let $\vec{x}$ be a fractional matching such that $x(D(e)) \leqslant \varepsilon^6$ for all $e \in E$. Then, there is a dynamic algorithm Sparsification$(\vec{x}, \varepsilon)$, that has the following properties.

(a) The algorithm maintains a subgraph $H \subseteq \text{supp}(\vec{x})$, such that $|E(H)| = O_\varepsilon(\mathsf{mwm}(\text{supp}(\vec{x})) \log n)$ and with high probability, $\mathsf{mwm}(H) \geqslant (1-\varepsilon) \cdot \sum_{e \in E} w(e) \cdot x(e)$.

(b) The algorithm handles the following updates to $\vec{x}$: the adversary can either remove an edge from $\text{supp}(\vec{x})$ or for any edge e, the adversary can reduce $x(e)$ to some new value $x(e) \geqslant 0$.

(c) The algorithm handles the above-mentioned updates in $\tilde{O}_\varepsilon(1)$ worst case time.

Proof. Note that (a) is implied by Theorem 88, and the fact that any fractional matching $\vec{y}$ with $y(e) \leqslant \varepsilon$ for all $e \in E$, satisfies odd set constraints for all odd sets of size at most $1/\varepsilon$. To see (b), note that deleting e from $\text{supp}(\vec{x})$ corresponds to just deleting e from G_i, and since our edge coloring algorithm is able to handle edge insertions and deletions in $\tilde{O}_\varepsilon(1)$ time, such updates can be handled in $\tilde{O}_\varepsilon(1)$ update time (see Lemma 82). Finally, if an update reduces $x(e)$ for some $e \in E$, then this corresponds to deleting e from some G_i and adding it to G_j for some $j < i$. Thus, the edge colouring algorithms running on G_i and G_j have to handle an edge insertion and deletion respectively, and this can be done in $\tilde{O}_\varepsilon(1)$ time (see Lemma 82). □

References

[ABD22] Sepehr Assadi, Aaron Bernstein, and Aditi Dudeja. Decremental matching in general graphs. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, 49th International Colloquium on Automata, Languages, and Programming, ICALP 2022, July 4-8, 2022, Paris, France, volume 229 of LIPIcs, pages 11:1–11:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[AKL19] Sepehr Assadi, Sanjeev Khanna, and Yang Li. The stochastic matching problem with (very) few queries. ACM Transactions on Economics and Computation (TEAC), 7(3):1–19, 2019.

[BCHN18] Sayan Bhattacharya, Deeparnab Chakrabarty, Monika Henzinger, and Danupon Nanongkai. Dynamic algorithms for graph coloring. In Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 1–20. SIAM, 2018.

[BDL21] Aaron Bernstein, Aditi Dudeja, and Zachary Langley. A framework for dynamic matching in weighted graphs. In Samir Khuller and Virginia Vassilevska Williams, editors, STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021, pages 668–681. ACM, 2021.

[BGS20] Aaron Bernstein, Maximilian Probst Gutenberg, and Thatchaphol Saranurak. Deterministic decremental reachability, scc, and shortest paths via directed expanders and congestion balancing. In 2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS), pages 1123–1134. IEEE, 2020.

[BHI15] Sayan Bhattacharya, Monika Henzinger, and Giuseppe F. Italiano. Deterministic fully dynamic data structures for vertex cover and matching. In Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '15, page 785–804, USA, 2015. Society for Industrial and Applied Mathematics.

[BK23] Joakim Blikstad and Peter Kiss. Incremental (1-ϵ)-approximate dynamic matching in o(poly(1/ϵ)) update time. In Inge Li Gørtz, Martin Farach-Colton, Simon J. Puglisi, and Grzegorz Herman, editors, 31st Annual European Symposium on Algorithms, ESA 2023, September 4-6, 2023, Amsterdam, The Netherlands, volume 274 of LIPIcs, pages 22:1–22:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

[BKS23a] Sayan Bhattacharya, Peter Kiss, and Thatchaphol Saranurak. Dynamic $(1+\epsilon)$-approximate matching size in truly sublinear update time. CoRR, abs/2302.05030, 2023.

[BKS23b] Sayan Bhattacharya, Peter Kiss, and Thatchaphol Saranurak. Dynamic algorithms for packing-covering lps via multiplicative weight updates. In Nikhil Bansal and Viswanath Nagarajan, editors, Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023, pages 1–47. SIAM, 2023.

[BS15] Aaron Bernstein and Cliff Stein. Fully dynamic matching in bipartite graphs. In Magnús M. Halldórsson, Kazuo Iwama, Naoki Kobayashi, and Bettina Speckmann, editors, Automata, Languages, and Programming - 42nd International Colloquium, ICALP 2015, Kyoto, Japan, July 6-10, 2015, Proceedings, Part I, volume 9134 of Lecture Notes in Computer Science, pages 167–179. Springer, 2015.

[BS16] Aaron Bernstein and Cliff Stein. Faster fully dynamic matchings with small approximation ratios. In Robert Krauthgamer, editor, Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016, pages 692–711. SIAM, 2016.

[CST23] Jiale Chen, Aaron Sidford, and Ta-Wei Tu. Entropy regularization and faster decremental matching in general graphs. 2023.

[Dah16] Søren Dahlgaard. On the hardness of partially dynamic graph problems and connections to diameter. CoRR, abs/1602.06705, 2016.

[DP09] Devdatt P. Dubhashi and Alessandro Panconesi. Concentration of Measure for the Analysis of Randomized Algorithms. Cambridge University Press, 2009.

[DP14] Ran Duan and Seth Pettie. Linear-time approximation for maximum weight matching. Journal of the ACM (JACM), 61(1):1–23, 2014.

[GHKT17] Andrew V. Goldberg, Sagi Hed, Haim Kaplan, and Robert E. Tarjan. Minimum-cost flows in unit-capacity networks. Theory Comput. Syst., 61(4):987–1010, 2017.

[GLS+19] Fabrizio Grandoni, Stefano Leonardi, Piotr Sankowski, Chris Schwiegelshohn, and Shay Solomon. $(1 + \epsilon)$-approximate incremental matching in constant deterministic amortized time. In Timothy M. Chan, editor, Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019, pages 1886–1898. SIAM, 2019.

[GP13] Manoj Gupta and Richard Peng. Fully dynamic (1+ e)-approximate matchings. 2013 IEEE 54th Annual Symposium on Foundations of Computer Science, pages 548–557, 2013.

[Gup14] Manoj Gupta. Maintaining approximate maximum matching in an incremental bipartite graph in polylogarithmic update time. In FSTTCS, 2014.

[HKNS15] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. Unifying and strengthening hardness for dynamic problems via the online matrix-vector multiplication conjecture. In Proceedings of the forty-seventh annual ACM symposium on Theory of computing, pages 21–30, 2015.

[JJST22] Arun Jambulapati, Yujia Jin, Aaron Sidford, and Kevin Tian. Regularized box-simplex games and dynamic decremental bipartite matching. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, 49th International Colloquium on Automata, Languages, and Programming, ICALP 2022, July 4-8, 2022, Paris, France, volume 229 of LIPIcs, pages 77:1–77:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[KPP16] Tsvi Kopelowitz, Seth Pettie, and Ely Porat. Higher lower bounds from the 3sum conjecture. In Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms, pages 1272–1287. SIAM, 2016.

[MMR94] Gradimir V. Milovanovic, Dragoslav S. Mitrinovic, and Themistocles M. Rassias. Topics in polynomials - extremal problems, inequalities, zeros. World Scientific, 1994.

[Sch03] Alexander Schrijver. Combinatorial optimization. polyhedra and efficiency. 2003.

[ST83] Daniel Dominic Sleator and Robert Endre Tarjan. A data structure for dynamic trees. J. Comput. Syst. Sci., 26(3):362–391, 1983.

[vdBCK+23] Jan van den Brand, Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. A deterministic almost-linear time algorithm for minimum-cost flow. CoRR, abs/2309.16629, 2023.

[Waj20] David Wajc. Rounding dynamic matchings against an adaptive adversary. In Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, pages 194–207, 2020.